

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DIPARTIMENTO DI INGEGNERIA INDUSTRIALE

Corso di laurea in

INGEGNERIA GESTIONALE

Progettazione e validazione di una pipeline multimodale per l'elaborazione di manuali tecnici industriali a supporto di sistemi RAG

Tesi di laurea in Impianti Industriali

Relatore
Prof. Mauro Gamberi

Correlatore
Ing. Pasquale Di Nardo

Presentata da
Michele Mazza

Sessione Ottobre 2026
Anno Accademico 2025/2026

A Boris

Abstract

Nell'assistenza alla conduzione e manutenzione, i sistemi di *Retrieval-Augmented Generation* (RAG) dipendono dalla preparazione dei manuali tecnici, le cui informazioni sono distribuite fra testo, immagini, tabelle e struttura. Nel progetto da cui nasce il lavoro, l'assistente per una macchina per elettroerosione usava un generatore *text-only*: ciò che era visibile solo nelle figure, come la posizione di un comando, restava escluso. La tesi progetta e valida una pipeline multimodale locale che trasforma manuali PDF eterogenei in una rappresentazione gerarchica comune e verificabile.

La pipeline instrada ogni documento verso tre profili: PDF digitale con indice incorporato, PDF digitale con struttura da ricostruire e scansione acquisita con OCR. Un modello visuale locale descrive le immagini, le tabelle sono serializzate con regole deterministiche e provenienza e controlli sono conservati in un fascicolo di esecuzione. Il corpus comprende due manuali, uno digitale in due varianti e uno scansionato; la valutazione combina prove integrali senza captioning e due esecuzioni finali su dieci pagine.

Le prove integrali elaborano 1112 pagine su 1112 per ciascuna variante digitale, con 14.047 elementi, e 34 su 34 per il manuale scansionato. Le esecuzioni finali superano 30 controlli tecnici su 30, restando sperimentali; delle 17 richieste di captioning, otto producono descrizioni ammissibili, cinque richiedono revisione e quattro superano il limite di lunghezza.

Strategie di acquisizione diverse convergono quindi in un output comune e tracciabile. Le descrizioni aggiungono informazioni utili, ma anche errori: una caption accettata conta dieci pulsanti dove l'immagine ne mostra dodici. La conformità tecnica non garantisce la correttezza semantica; l'effetto sul RAG richiede una valutazione distinta.

Indice

Abstract **i**

1 Introduzione **1**
1.1 Assistenza industriale e origine del lavoro 1
1.2 LLM e sistemi RAG 3
1.3 Ingestion di manuali eterogenei e rappresentazione comune 5
1.4 Contenuti multimodali e arricchimento documentale 5
1.5 Stato dell'arte 6
1.6 Contributo, perimetro e domande di ricerca 7

2 Materiali e metodi **8**
2.1 Caso applicativo, corpus e requisiti 8
2.2 Architettura modulare e orchestrazione del workflow 10
2.3 Detection e strategie di acquisizione 12
2.4 Modello documentale canonico e assemblaggio gerarchico 14
2.5 Gestione degli asset e arricchimento multimodale 17
2.6 Output, tracciabilità e controlli della pipeline 20
2.7 Metodo di valutazione 23

3 Risultati e discussione **29**
3.1 Esiti della pipeline integrata per profilo 29
3.2 Qualità della rappresentazione comune 31
3.3 Esiti dell'arricchimento multimodale 34
3.4 Esiti dei controlli, fallimenti e tracciabilità 36
3.5 Discussione, limiti e trasferibilità 38

4 Conclusioni **40**
4.1 Sintesi del contributo 40
4.2 Risposte alle domande di ricerca 40
4.3 Sviluppi futuri 41

Nota sull'utilizzo dei modelli di linguaggio **44**

Codice sorgente **45**

Bibliografia **46**

Capitolo 1

Introduzione

Questa tesi nasce nel filone di ricerca di Pasquale Di Nardo sull'impiego di assistenti basati su intelligenza artificiale nelle attività industriali. Le criticità osservate nell'uso di un assistente per una macchina per elettroerosione hanno orientato il lavoro verso la preparazione della conoscenza documentale. Il contributo sviluppato è una pipeline di ingestion integrata nell'esecuzione e modulare nella progettazione: a partire da manuali PDF eterogenei, organizza testo, immagini e tabelle in una rappresentazione gerarchica comune, sulla quale operano le successive fasi di arricchimento e verifica.

1.1 Assistenza industriale e origine del lavoro

Le attività di conduzione e manutenzione, indicate nel seguito come *operation e maintenance* (O&M), richiedono di collegare informazioni tecniche e azioni operative. Interpretare un allarme, individuare un componente o eseguire una procedura significa consultare conoscenze distribuite tra manuali, istruzioni ed esperienza del personale. La complessità dei macchinari rende quindi rilevante non soltanto la disponibilità della documentazione, ma anche la possibilità di utilizzarla durante il lavoro (Ludwig et al. 2025).

Gli assistenti digitali offrono un accesso conversazionale a questa conoscenza. Il valore del supporto dipende però dalla sua utilizzabilità nel compito: una spiegazione corretta in termini generali può non bastare a eseguire un'operazione. Per questo la valutazione degli assistenti industriali considera sia il comportamento del sistema sia l'interazione con l'utente (Colabianchi et al. 2024).

In questo contesto si colloca il progetto di ricerca di Di Nardo. Nei primi colloqui di supervisione, il progetto è stato presentato a partire da un assistente basato su modelli linguistici e recupero della conoscenza dai manuali, destinato alle attività di O&M di una macchina per elettroerosione. Il sistema è eseguito *on-premise*, all'interno dell'organizzazione, senza inviare la documentazione industriale a servizi esterni. Impiega un'architettura *Retrieval-Augmented Generation* (RAG), introdotta nella Sezione 1.2.

Nella configurazione utilizzata per l'esperimento, il generatore era Qwen3-4B, un modello *text-only* della famiglia Qwen3 (Qwen Team 2025). Riceveva la domanda e i passaggi testuali recuperati, non le immagini del manuale. Questa configurazione costituisce il punto di partenza del lavoro, non una dimostrazione di superiorità rispetto alle alternative multimodali.

La campagna sperimentale del progetto si è evoluta da quesiti a risposta multipla all'interazione con un Digital Twin: un ambiente che riproduce l'interfaccia della macchina e permette ai partecipanti di affrontare compiti operativi in simulazione. Il confronto tra uso dell'assistente e consultazione del manuale considera tempi, accuratezza, usabilità mediante System Usability Scale (SUS), carico di lavoro percepito mediante NASA-TLX e situazioni classificate come *critical*

failures, rilevanti per la sicurezza o la qualità del prodotto. La Figura 1.1 riassume le condizioni di supporto e le dimensioni osservate.

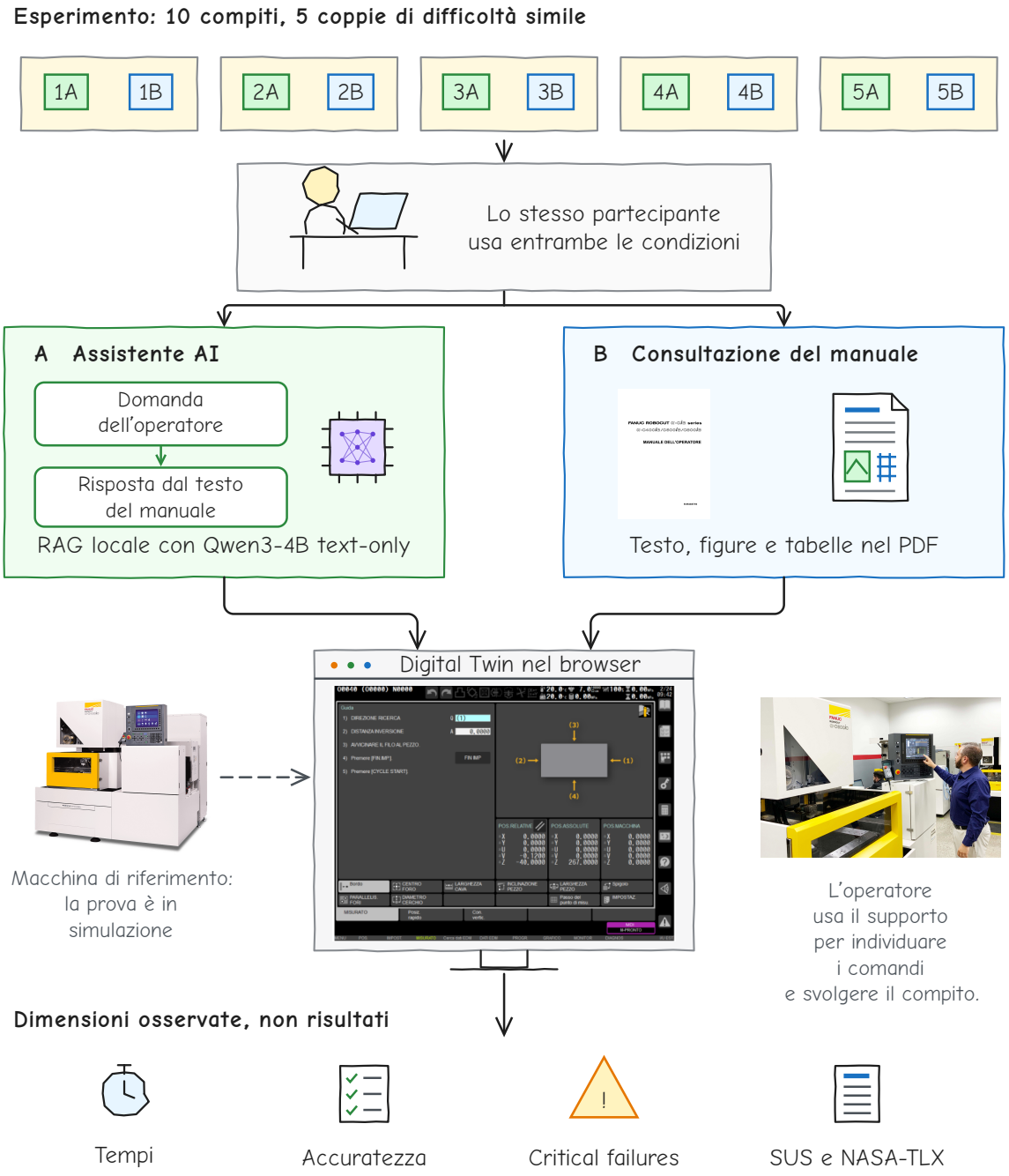


Figura 1.1: Ricostruzione del progetto di Pasquale Di Nardo a partire dallo schema sperimentale fornito dall'autore della tesi. La schermata del Digital Twin è ritagliata dalla stessa figura; macchina, copertina e immagine di interazione con il touch panel provengono dai materiali allegati. Le vignette di dialogo sono schematiche, non risposte registrate. A e B indicano condizioni dello stesso partecipante, non gruppi distinti né un ordine di esecuzione. Questa sperimentazione non è la validazione finale della pipeline.

La prova dell'assistente in laboratorio ha offerto un'occasione concreta per discutere il passaggio dalla spiegazione all'azione. Il sistema poteva fornire indicazioni generali senza aiutare a localizzare un tasto nell'interfaccia o a identificarlo rispetto ai comandi vicini. Nel dialogo con il supervisore, questa distanza è stata riassunta nell'immagine del modello come «gran teorico». È emersa così una risorsa non utilizzata: le immagini del manuale, nelle quali una posizione può essere visibile senza essere descritta nel testo.

L'osservazione ha orientato la scelta di approfondire la componente tecnica anziché riprogettare l'esperimento sugli operatori, individuando un limite sul quale intervenire senza attribuire tutti gli errori all'assenza delle immagini. Il successivo confronto ha ampliato il problema: acquisire le figure non basta, se si perdono la gerarchia del manuale e i rapporti tra contenuti. La direzione multimodale ha quindi dato origine a un lavoro di ingestion e strutturazione documentale, nel quale l'enrichment è uno stadio e non l'intero sistema.

1.2 LLM e sistemi RAG

I *Large Language Models* (LLM) sono modelli linguistici addestrati su grandi quantità di dati. Nei modelli generativi autoregressivi, il testo viene prodotto progressivamente sulla base della sequenza disponibile, che può includere la richiesta dell'utente e informazioni aggiuntive. La conoscenza appresa nei parametri non coincide però con un archivio aggiornato dei documenti di un'organizzazione: per rispondere su un manuale specifico è utile fornire al modello le fonti pertinenti (Lewis et al. 2020). La Figura 1.2 illustra il ciclo di generazione.

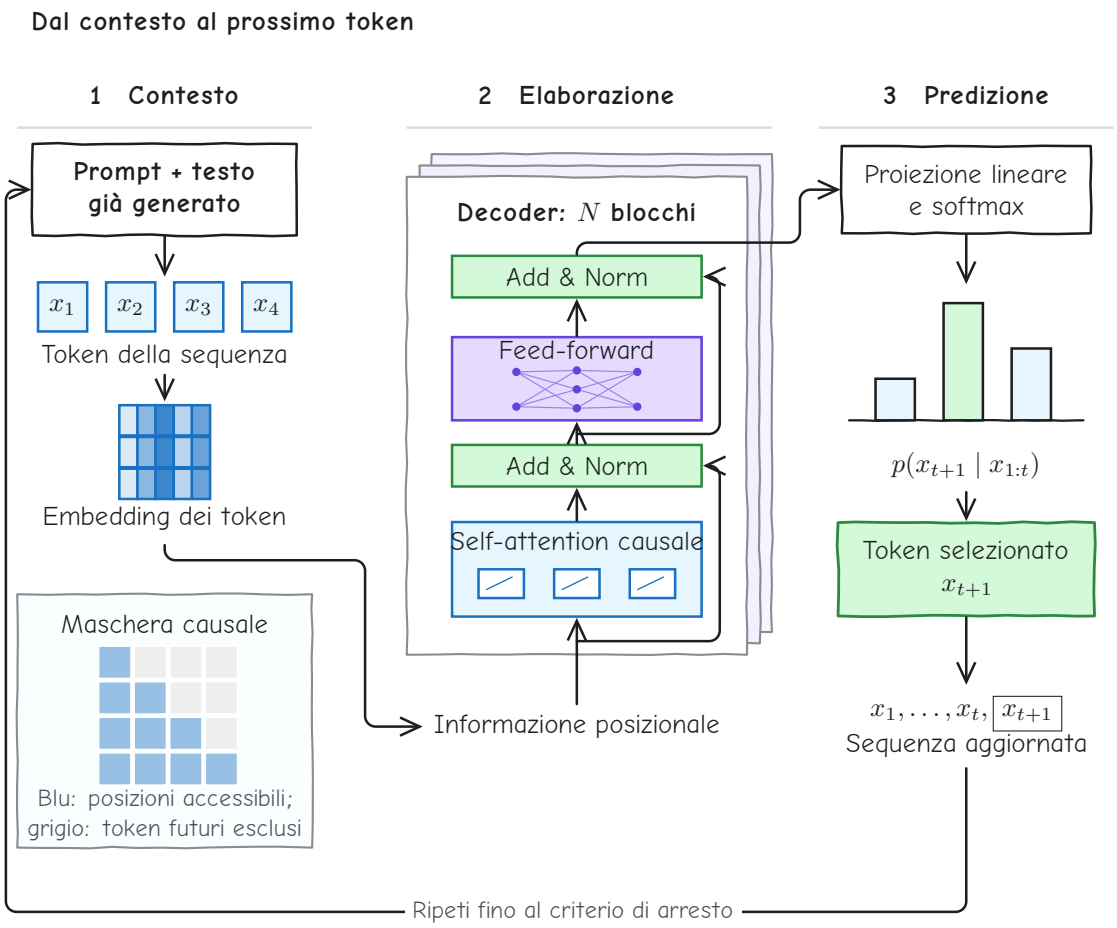


Figura 1.2: Rielaborazione didattica ispirata alle Figure 1 e 2 di Vaswani et al. (2017): stack di attenzione, connessioni residue e distribuzione del prossimo token. Qui è rappresentato un decoder-only, senza encoder né cross-attention; l'ordine delle normalizzazioni è schematico e non descrive l'implementazione specifica di Qwen.

Il paradigma RAG combina generazione linguistica e recupero da una base informativa esterna. Durante la preparazione, i documenti vengono elaborati, suddivisi in unità dette *chunk* e indicizzati. Nel recupero vettoriale, gli *embedding* sono rappresentazioni numeriche che permettono

di confrontare domanda e contenuti. Quando arriva una richiesta, il sistema recupera le unità pertinenti e le inserisce nel contesto del generatore (Lewis et al. 2020; Amugongo et al. 2025). La Figura 1.3 distingue preparazione e interrogazione e colloca il contributo della tesi prima del chunking.

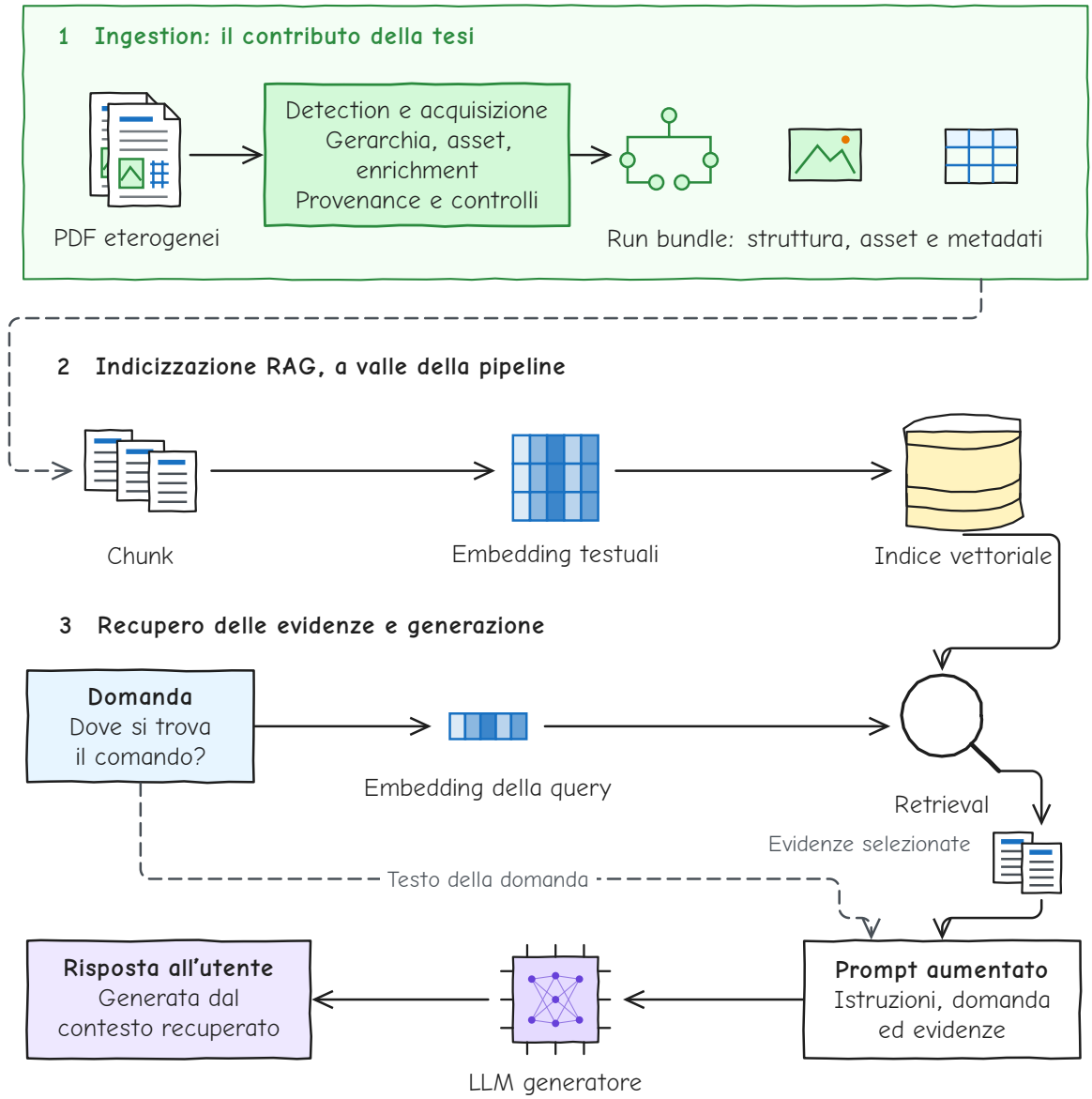


Figura 1.3: Schema originale ispirato alla separazione retrieval/generazione di Lewis et al. (2020) e ai flussi testo/immagini della Figura 1 di Riedler e Langer (2024). Qui è illustrato il percorso con rappresentazioni testuali: non una replica delle architetture dei paper né dell'assistente sperimentale. Solo la fascia verde appartiene al contributo implementato e validato della tesi.

L'accesso a fonti esterne non garantisce la correttezza dell'output. Il recupero può selezionare contenuti non pertinenti e il generatore può interpretarli erroneamente. Inoltre, un errore può precedere entrambe le fasi: se un'informazione è stata persa nella preparazione del documento, non sarà disponibile nell'indice (Amugongo et al. 2025).

In questa tesi, *ingestion documentale* indica l'acquisizione e la trasformazione dei manuali fino alla produzione di una rappresentazione strutturata, corredata da provenienza e controlli. Il confine precede il chunking e permette di valutarne gli esiti senza assumere una specifica strategia

di recupero o generazione. Anche un futuro assistente multimodale richiederebbe fonti acquisite e contestualizzate: cambiare il generatore non elimina il problema documentale.

1.3 Ingestion di manuali eterogenei e rappresentazione comune

Un manuale tecnico non è una semplice sequenza di frasi. Capitoli e sottocapitoli delimitano l'ambito delle istruzioni; figure e tabelle completano descrizioni che, isolate, possono risultare insufficienti. La rappresentazione deve quindi conservare sia i contenuti sia le relazioni che ne permettono l'interpretazione (Auer et al. 2024).

Occorre distinguere due perdite possibili. Un trattamento text-only esclude l'informazione disponibile soltanto nelle immagini; una rappresentazione piatta può conservare le parole ma indebolirne i collegamenti con sezioni e altri elementi. Includere le figure non ricostruisce automaticamente la gerarchia, così come riconoscere i capitoli non rende accessibile il significato di un diagramma. Il *document understanding* affronta il riconoscimento degli elementi e della loro organizzazione: OCR, analisi del layout e riconoscimento tabellare svolgono compiti distinti all'interno di questa trasformazione (Auer et al. 2024).

Per elaborare un PDF occorre sia acquisirne i contenuti sia individuare la struttura di capitoli e sottocapitoli. La pipeline riconosce le caratteristiche del file (*detection*) e seleziona il percorso adatto (*routing*). Il profilo `digital_outline` usa l'indice strutturato incorporato nel PDF, detto *outline*. Il profilo `digital_reconstructed` lavora invece su un PDF digitale privo di outline e ne ricostruisce l'indice gerarchico (TOC) a partire dai segnali presenti nel documento. Il profilo `scanned_ocr` acquisisce i contenuti mediante OCR; per la gerarchia usa l'outline, se disponibile, oppure la ricostruisce. Acquisire i contenuti tramite OCR e ricostruire la struttura restano quindi due compiti distinti.

I tre percorsi fanno parte dello stesso workflow e producono un documento organizzato secondo il modello comune `ManualDocument`. In questo modello, testo, immagini e tabelle sono collegati alle pagine e alle sezioni di appartenenza: acquisire o ricostruire la TOC serve a costruire e conservare tali relazioni. Le fasi successive, tra cui l'arricchimento delle immagini e la serializzazione delle tabelle, lavorano sullo stesso modello, qualunque sia il profilo utilizzato. Solo dopo queste lavorazioni il documento viene salvato nel bundle come `manual.json`. I dettagli implementativi sono descritti nel Capitolo 2.

1.4 Contenuti multimodali e arricchimento documentale

Le immagini possono indicare dove si trova un componente, come sono collegati due elementi o quale aspetto presenta una schermata. Le tabelle dipendono dalle associazioni tra righe, colonne e intestazioni: conservare i valori senza queste relazioni può alterarne il significato. Acquisizione, organizzazione e contestualizzazione degli asset sono pertanto necessarie anche prima di applicare un modello generativo (Auer et al. 2024; Charan et al. 2026).

L'enrichment aggiunge un livello interpretativo alla rappresentazione. I *Vision-Language Models* (VLM) elaborano immagini e testo e possono produrre descrizioni linguistiche del contenuto visuale, anche utilizzandone il contesto documentale. Il *captioning* permette così di rendere parte di tale informazione utilizzabile attraverso strumenti di ricerca testuale (Riedler e Langer 2024).

Nel sistema sviluppato, l'enrichment è un modulo separato dall'acquisizione e dall'assemblaggio, che opera sul documento canonico tramite un'interfaccia di caption provider. Il provider concreto utilizzato nello studio impiega un modello locale per descrivere le immagini. La serializzazione delle tabelle ha invece una responsabilità distinta: organizza i dati già estratti secondo regole fisse, senza un modello generativo. Il captioning può essere disattivato per controllare gli altri

passaggi; queste esecuzioni diagnostiche non verificano però il processo completo e vanno distinte da quelle con captioning attivo.

Una descrizione generata può rendere esplicite informazioni utili presenti nell'immagine, ma può anche omettere dettagli o interpretarli in modo errato. Non deve quindi sostituire la fonte originale: occorre conservare entrambe e mantenere il collegamento con la pagina e la sezione del manuale da cui provengono. La valutazione della pipeline deve perciò verificare non solo la struttura del documento e i collegamenti tra i suoi elementi, ma anche la correttezza e l'utilità delle descrizioni aggiunte.

1.5 Stato dell'arte

Questa rassegna esamina alcuni lavori utili a capire come preparare i contenuti dei manuali per un assistente industriale. Il confronto segue quattro aspetti: quali operazioni di acquisizione sono automatiche, come viene conservata la struttura del documento, come sono trattati immagini e tabelle e che cosa viene valutato. L'obiettivo è collocare il lavoro della tesi rispetto agli studi selezionati, non fornire una revisione sistematica dell'intera letteratura.

Un primo gruppo di studi riguarda l'impiego degli assistenti nelle attività industriali. Colabianchi et al. (2024) valutano l'interazione con l'assistente e le prestazioni nelle attività di assemblaggio. Ludwig et al. (2025) studiano invece un assistente per la manutenzione, concentrandosi sul recupero di informazioni da un'ontologia e sulla qualità delle risposte. Altri lavori affrontano l'esecuzione locale dei sistemi, sia per l'assistenza alle macchine utensili sia per la consultazione di documenti nel settore energetico (Cho et al. 2026; Lee et al. 2025). Questi studi aiutano a definire il contesto applicativo della tesi. Valutare l'assistente nel suo insieme, però, non permette necessariamente di stabilire quanto i risultati dipendano dalla preparazione dei documenti.

Un secondo gruppo di lavori si concentra sulla struttura dei manuali e sui collegamenti tra i loro contenuti. Nam et al. (2025) organizzano la gerarchia del documento e le associazioni tra testo e immagini, ricorrendo anche a interventi manuali. Knollmeyer et al. (2025) usano invece la struttura documentale per costruire un grafo a supporto del recupero delle informazioni. È quindi importante distinguere due attività: ricostruire e conservare le relazioni tra i contenuti, come avviene nell'ingestion, e usare tali relazioni per cercare informazioni, come avviene in un sistema GraphRAG. Per confrontare questi approcci con la pipeline della tesi, occorre chiarire quali passaggi sono automatici, quali richiedono intervento umano e quali appartengono al successivo recupero delle informazioni.

La qualità dell'estrazione resta un problema anche quando si migliora il recupero. Krainovskikh e Samigulin (2026) riportano errori dovuti a tabelle e diagrammi non acquisiti dall'estrazione testuale e a riferimenti distribuiti tra sezioni diverse. Il problema può quindi nascere prima della ricerca: se un contenuto manca o perde il collegamento con il proprio contesto, il sistema dispone di una base informativa incompleta. Per questo la verifica dell'ingestion deve controllare sia i contenuti estratti sia la gerarchia e le associazioni tra gli elementi, non soltanto il formato dell'output.

Per le immagini, Riedler e Langer (2024) confrontano due modi di rappresentarne il contenuto ai fini del recupero: embedding multimodali e sintesi testuali. Nelle loro prove i risultati sono simili, con un lieve vantaggio delle sintesi nella maggior parte delle metriche. Le sintesi servono però a trovare le immagini originali, che vengono poi fornite al generatore: il confronto riguarda quindi il recupero delle immagini, non una presunta superiorità dei generatori che ricevono soltanto testo. Anche Amaram et al. (2026) trasformano le figure in descrizioni linguistiche, ma segnalano passaggi di acquisizione ancora manuali. Questi lavori mostrano perché occorre considerare insieme la rappresentazione delle immagini e il processo necessario per ottenerla.

Nel complesso, gli studi esaminati offrono soluzioni per acquisire, organizzare e utilizzare i contenuti dei manuali, ma non sempre valutano separatamente la qualità dei passaggi documentali. La tesi si concentra su questo livello: integra strategie di acquisizione diverse in un'unica pipeline e verifica la rappresentazione prodotta prima del suo impiego in un sistema RAG. Il contributo non consiste nell'introdurre singolarmente gerarchia, multimodalità o esecuzione locale, ma nel combinarle e valutarne il funzionamento sui manuali dello studio.

1.6 Contributo, perimetro e domande di ricerca

L'obiettivo della tesi è progettare e validare una pipeline che prepari manuali tecnici industriali per un successivo impiego in sistemi RAG. Il contributo consiste nell'integrare il processo dal PDF al bundle finale: riconoscere le caratteristiche del documento, scegliere il percorso di acquisizione e ricostruire o conservare la TOC e la gerarchia. Il contratto documentale comune separa i profili dalle lavorazioni successive, con responsabilità distinte per arricchimento, serializzazione, registrazione della provenienza, controlli e pubblicazione.

L'interfaccia a riga di comando (CLI) riceve il PDF e invoca l'orchestratore del backend, che coordina le fasi di elaborazione. Il nucleo dell'ingestion è separato dalla CLI: questa permette di utilizzarlo, non coincide con la pipeline né costituisce un contributo scientifico autonomo. Il risultato è un *run bundle*, un insieme di file con documento strutturato, asset estratti, metadati ed esiti dei controlli. Le informazioni di indice incluse descrivono e organizzano i contenuti del bundle: non costituiscono l'indice usato da un sistema RAG per cercare le risposte.

Come mostra la Figura 1.3, il lavoro si ferma alla preparazione dei documenti. Restano esclusi la suddivisione in chunk, il calcolo degli embedding, l'indicizzazione RAG, il recupero delle informazioni e la generazione delle risposte. Dopo lo sviluppo della pipeline non è stato ripetuto il confronto con gli operatori: non si può quindi affermare che il nuovo trattamento dei manuali migliori le prestazioni dell'assistente. Tempi, accuratezza, SUS, NASA-TLX e critical failures riguardano l'esperimento che ha motivato la tesi, non la validazione qui condotta.

La validazione verifica invece come la pipeline elabora i diversi PDF e quale qualità hanno i documenti prodotti. Vengono esaminati i contenuti acquisiti, la gerarchia, i collegamenti tra gli elementi, le descrizioni aggiunte e la possibilità di risalire alle fonti e agli esiti dei controlli. Si distinguono due aspetti: la conformità tecnica, ossia il rispetto dei requisiti di formato e coerenza dell'output, e la correttezza semantica, cioè la fedeltà dei contenuti e delle loro associazioni al manuale originale. Superare i controlli tecnici non basta a garantire la seconda: anche errori e limiti devono entrare nella valutazione.

Il lavoro risponde quindi a tre domande di ricerca:

RQ1 – Ingestion integrata. Come ottenere una rappresentazione gerarchica comune da manuali PDF eterogenei, usando strategie di acquisizione diverse? Quali limiti emergono nel conservare i contenuti e i collegamenti tra gli elementi?

RQ2 – Arricchimento. Rispetto ai contenuti già estratti, quali informazioni aggiunge l'arricchimento multimodale locale e quali omissioni o errori introduce?

RQ3 – Verificabilità. Come usare le informazioni di provenienza e i controlli per verificare i risultati della pipeline, distinguendo la conformità tecnica dalla correttezza semantica?

Le risposte valgono per i documenti e le esecuzioni analizzati nello studio. Il Capitolo 2 descrive la pipeline e il metodo di valutazione; il Capitolo 3 presenta e discute i risultati; il Capitolo 4 risponde alle domande di ricerca e distingue i possibili miglioramenti della pipeline dalle future verifiche del suo impiego in un sistema RAG.

Capitolo 2

Materiali e metodi

Questo capitolo descrive i materiali, l'architettura e il metodo di verifica della pipeline di ingestion sviluppata nella tesi. La descrizione e la valutazione si basano su tre fonti complementari: il codice del pacchetto `manual_ingestion` e i test ne definiscono e verificano il comportamento; la documentazione ricostruisce le scelte progettuali e le prove di accettazione; i bundle e le registrazioni delle due esecuzioni finali di enrichment del 15 settembre 2026 consentono di controllare gli ingressi e le uscite di quel ciclo sperimentale.

Il progetto separa la distribuzione del software dai materiali dello studio. La repository pubblica `industrial-manual-ingestion` contiene il prodotto finale; il workspace privato `tesi-di-nardo` raccoglie il diario delle decisioni, le matrici di accettazione, le run V4 e i materiali di valutazione. È una distinzione organizzativa, che non modifica l'architettura della pipeline né il contratto del run bundle. Nel seguito, i riferimenti ai moduli sono relativi al pacchetto `manual_ingestion`, collocato nella directory `src/manual_ingestion/` della repository pubblica.

Le sigle di versione indicano oggetti diversi: V2 designa l'architettura integrata del prodotto, mentre V3 e V4 identificano cicli dello studio di enrichment. Non corrispondono quindi al tag della release pubblica `v1.0.0`, né alla versione del pacchetto registrata nelle run o a quella dello schema documentale. Anche il prompt ha una propria versione: il ciclo V4 usa ancora `technical-caption-v3`.

2.1 Caso applicativo, corpus e requisiti

Il caso applicativo è la preparazione di manuali tecnici industriali per una successiva consultazione assistita, nel contesto delineato nel Capitolo 1. L'unità di ingresso della pipeline è un PDF; l'unità di uscita è un insieme coordinato di artefatti, denominato *run bundle*, che descrive sia il documento elaborato sia l'esecuzione che lo ha prodotto. Il confine del sistema precede il chunking: la pipeline non calcola embedding, non costruisce un indice vettoriale e non esegue retrieval o generazione di risposte.

L'eterogeneità degli ingressi è affrontata distinguendo tre condizioni operative: PDF con testo digitale e outline incorporato; PDF con testo digitale ma privi di outline; PDF con copertura testuale insufficiente, che richiedono OCR. Si tratta di condizioni rilevate dal programma, non di tre corpus indipendenti. In particolare, un PDF misto può essere instradato verso l'OCR anche se alcune pagine contengono testo estraibile. L'assenza di outline non implica inoltre che qualsiasi struttura sia ricostruibile: il percorso digitale senza segnalibri richiede un indice stampato riconoscibile.

L'implementazione risponde a quattro requisiti. Il primo è uniformare l'uscita dei parser senza perdere l'identità dei contenuti acquisiti. Il secondo è conservare la relazione tra elementi, pagine e capitoli, insieme alle coordinate quando disponibili. Il terzo è separare le informazioni estratte da

quelle generate e rendere esplicite le esclusioni. Il quarto è pubblicare soltanto bundle tecnicamente coerenti, distinguendo quelli conformi alla politica di accettazione da quelli sperimentali. Il campo `include_in_rag` esprime una destinazione d'uso prevista, mentre l'indicizzazione dell'elemento appartiene al sistema a valle. A questi requisiti si affiancano l'esecuzione locale, l'assenza di fine-tuning e un output strutturato che non dipende dal sistema Django/RAG a valle, come documentato nel diario di progetto.

Documenti e condizioni di ingresso

La matrice di accettazione identifica tre file di ingresso, riportati nella Tabella 2.1.[1] I primi due rappresentano lo stesso manuale dell'operatore FANUC ROBOCUT della serie Alpha-CiB, relativo ai modelli Alpha-C400iB, C600iB e C800iB: il secondo file è la variante priva dell'outline incorporato, ma con testo digitale conservato. Il terzo è un manuale scansionato del tornio WM250, distinto per contenuto e famiglia di macchina. Il corpus comprende dunque tre condizioni di ingresso, ma due manuali di base; le due versioni digitali non sono osservazioni indipendenti.

Tabella 2.1: Tre file di ingresso, corrispondenti a due manuali di base. Fonte: matrice di accettazione; gli hash completi sono conservati nella fonte. I due PDF digitali sono varianti dello stesso manuale.

Condizione	File, pagine e hash abbreviato	Evidenza di routing
Digitale con outline	`Manuale dell'operatore.pdf` 1112 pagine SHA-256: `5a399fe1a362...`	963 segnalibri; testo utile in 6 delle 7 pagine campionate.
Digitale senza outline	`Manuale no TOC si OCR.pdf` 1112 pagine SHA-256: `cba3d897cb47...`	Nessun segnalibro; testo utile in 6 delle 7 pagine campionate.
Scansione	`Manuale no TOC no OCR.pdf` 34 pagine SHA-256: `12bdfe09377c...`	Nessun segnalibro; testo utile in 0 delle 7 pagine campionate.

I nomi storici dei file non descrivono operazioni eseguite dalla pipeline: l'espressione "si OCR" nel nome del secondo PDF indica la disponibilità di testo, ma il profilo digitale usa Docling senza OCR. Nelle pagine digitali conservate il testo è italiano; nelle pagine WM250 il testo acquisito è inglese. Entrambe le run finali sono state lanciate con `-language it`: questo valore è un parametro del documento di uscita impostato al lancio, anziché il risultato di un riconoscimento automatico della lingua; nel caso WM250 differisce quindi dalla lingua della fonte.

Il confronto fra i due PDF digitali consente di studiare la ricostruzione della TOC usando l'outline della variante originale come riferimento. Non costituisce invece un confronto controllato fra OCR e acquisizione digitale: il manuale WM250 differisce anche per contenuto, dimensione e layout. Gli input misti e i digitali senza indice stampato rientrano nei limiti operativi discussi nel codice, ma non sono ulteriori casi empirici del corpus documentato.

Copertura delle prove e materiali disponibili

Le prove di accettazione del luglio 2026 combinano elaborazioni complete senza provider e selezioni ridotte con enrichment. Per lo schema 1.1 la matrice registra i run `case1-schema11-full-no-enrich-20260712` e `case2-schema11-full-no-enrich-20260712`, entrambi su 1112 pagine, e `case3-schema11-full-no-enrich-20260712` sulle 34 pagine WM250. Per i due casi digitali documenta inoltre selezioni delle pagine 3 e 1004, comprendenti tabelle.

[1] Fonte dell'inventario: `tesi-di-nardo/docs/acceptance-matrix.md`, sezione "Input reali e routing". La relazione tra i due PDF digitali e l'identificazione del manuale WM250 sono documentate anche in `tesi-di-nardo/docs/toc-reconstruction-plan.md`.

Queste prove verificano aspetti complementari: le run integrali la copertura e l'assemblaggio, le selezioni arricchite l'integrazione con contenuti visuali reali. La copertura del captioning resta limitata alle selezioni ridotte.

Il ciclo finale V4 del 15 settembre 2026 comprende invece due sole esecuzioni selettive, una digitale con outline e una OCR, archiviate in `tesi-di-nardo/enrichment-v4-final-20260915`. La composizione dettagliata è descritta nella Sezione 2.7. Il percorso digitale ricostruito non è ripetuto in questo ciclo per non duplicare le immagini del medesimo manuale. La copertura dei tre profili appartiene pertanto alle prove della pipeline; la valutazione finale dell'enrichment riguarda due profili e non rappresenta tre esperimenti indipendenti.

L'archivio dei materiali di tesi nel workspace privato `tesi-di-nardo` conserva sorgenti e test archiviati a supporto dei riscontri documentati, la documentazione dello studio e i due bundle V4 con le relative evidenze. La repository pubblica `industrial-manual-ingestion` raccoglie invece il prodotto software finale, con il pacchetto, la CLI, i test, il viewer, un esempio sintetico e la documentazione d'uso. Nel seguito, i riferimenti all'archivio del progetto e ai materiali non inclusi riguardano l'archivio della tesi, non il contenuto della repository pubblica. Non sono inclusi i tre PDF integrali, i bundle storici di luglio né il rapporto qualitativo curato richiamato dai documenti sotto `outputs/enrichment-evaluation`. Per questo, nella valutazione si riportano i dati storici dalla matrice di accettazione e si riservano le verifiche dirette agli artefatti V4 conservati. Le note didattiche in `docs/learning` e i piani del prototipo documentano l'evoluzione delle scelte, anziché le specifiche correnti: contengono anche componenti successivamente rimosse, come `rag_index.json`, o confronti fra motori non esposti dalla CLI finale.

2.2 Architettura modulare e orchestrazione del workflow

Il backend coordina un unico workflow tramite `orchestrator.py`; il registro in `profiles/registry.py` costruisce il profilo selezionato dalla detection. I profili condividono il contratto `ProfileResult`, che comprende il modello canonico `ManualDocument` e la TOC. Le lavorazioni comuni operano sul documento così assemblato, separatamente dalle strategie di acquisizione; `manual.json` ne è la rappresentazione persistita nel bundle dopo tali lavorazioni. La modularità riguarda questi confini e la separazione dell'enrichment, dell'interfaccia del caption provider e della CLI, non una libera sostituibilità di ogni componente.

La CLI in `cli.py` è un'interfaccia al backend, non il nucleo della pipeline né un contributo metodologico autonomo. Espone `detect` per ispezionare il PDF, `ingest` per invocare l'orchestratore, `validate` per verificare un bundle senza modificarlo e `schema` per esportare lo schema JSON del documento o del manifest. Non offre selettori manuali del parser o della strategia strutturale. Le opzioni `-pages` e `-no-enrich` consentono una selezione diagnostica delle pagine e la disattivazione del captioning: non equivalgono all'elaborazione ordinaria completa.

L'ordine effettivo delle operazioni è il seguente:

1. **Preparazione e detection.** Sono verificati sorgente, destinazione, identificativo della run e pagine richieste. Vengono acquisiti dimensione, impronta SHA-256 e proprietà del file; si rilevano quindi le capacità del PDF. Se è configurato il provider delle descrizioni, il controllo preliminare del servizio e del modello precede l'estrazione.

2. **Acquisizione e assemblaggio.** Il profilo lavora in una directory temporanea adiacente alla destinazione. Produce un `ManualDocument`, una lista di voci `TocEntry`, avvisi ed eventuali dettagli diagnostici. L'orchestratore verifica la coerenza del risultato con input, profilo e pagine richieste.

3. **Trattamento comune degli elementi visuali.** Sul documento assemblato si applica il filtro dei ritagli piccoli; segue il captioning delle immagini ammissibili, oppure la gestio-

ne esplicita della modalità diagnostica senza provider. Sono registrate le decisioni e le informazioni sul runtime.

4. **Serializzazione delle tabelle.** Le tabelle vengono trasformate in righe testuali contestualizzate con una procedura deterministica, distinta dalle chiamate al modello visuale.

5. **Controlli e pubblicazione.** Vengono costruiti manifest e valutazione di promozione, scritti gli artefatti provvisori e verificata la loro coerenza. Il manifest finale assume lo stato `validated` oppure `experimental`; prima della pubblicazione si ricontrolla che il sorgente non sia cambiato.

La Figura 2.1 rappresenta questa convergenza logica. I ritagli vengono già creati negli adattatori durante l'acquisizione; ciò che segue l'assemblaggio è il loro trattamento comune, non la loro prima estrazione. Analogamente, nel percorso digitale ricostruito l'analisi dell'indice precede il parsing dei contenuti con Docling, mentre nel percorso OCR la ricostruzione utilizza gli elementi già riconosciuti.

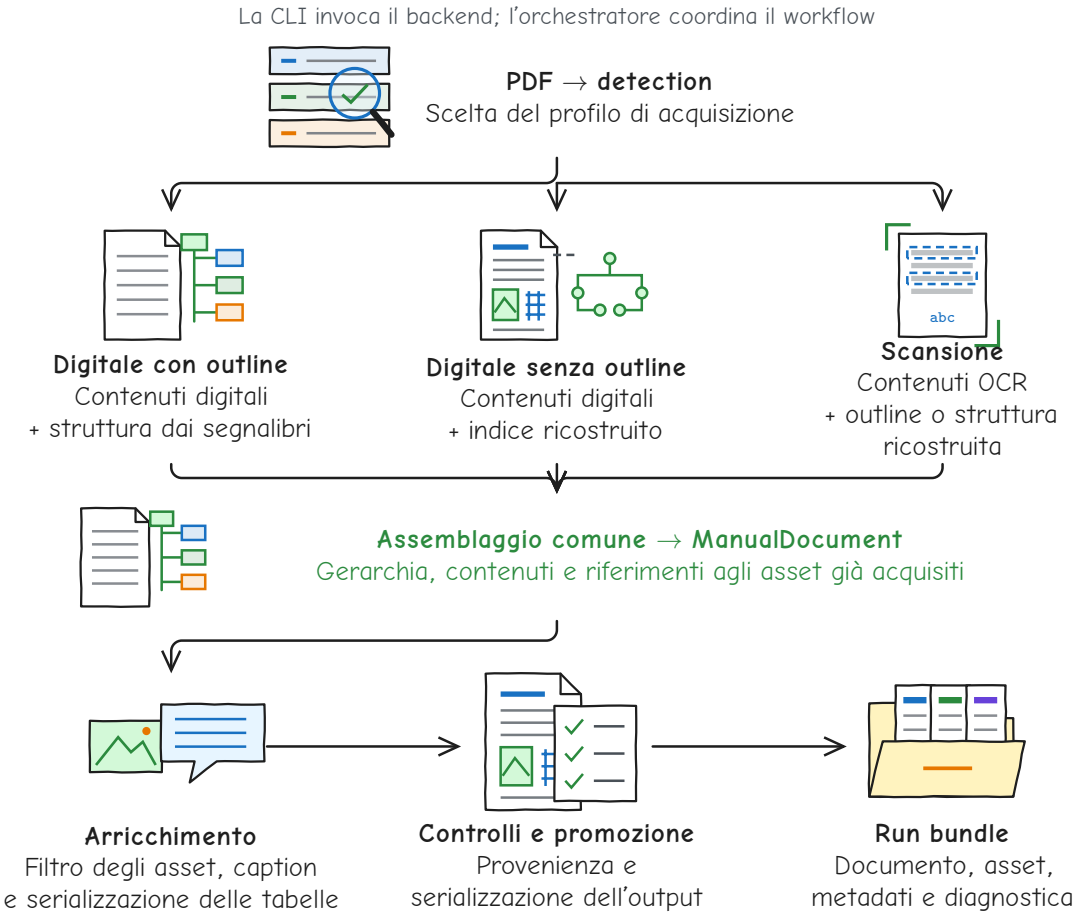


Figura 2.1: Tre percorsi di acquisizione convergono sullo stesso contratto documentale e sulle stesse lavorazioni successive. Pagine, gerarchie e fascicolo rappresentano i diversi artefatti. Vista logica di `manual_ingestion`, non grafo delle chiamate: l'OCR acquisisce contenuti, non determina da solo la gerarchia.

2.3 Detection e strategie di acquisizione

Segnali e regole di instradamento

La detection, implementata in `detection.py`, usa PyMuPDF per controllare l'apertura del PDF, la presenza di pagine, l'eventuale protezione mediante password, il numero di voci dell'outline e la disponibilità di testo. Il campionamento è deterministico e comprende al massimo sette pagine distribuite sull'intero documento, inclusi gli estremi. Indicando con N il numero totale di pagine e con $m = \min(N, 7)$ la dimensione del campione, per $m > 1$ le pagine campionate sono

$$p_j = 1 + \left\lfloor \frac{j(N-1)}{m-1} \right\rfloor, \qquad j = 0, \ldots, m-1. \tag{2.1}$$

Per un documento di una sola pagina viene esaminata quella pagina. Una pagina offre evidenza testuale utilizzabile quando contiene almeno 40 caratteri non bianchi. Il testo digitale è considerato sufficientemente diffuso se il numero di pagine che soddisfano tale requisito è almeno $\lceil 0{,}75m \rceil$.

Se questa soglia è raggiunta e l'outline contiene almeno una voce, viene selezionato `digital_outline`; se la copertura è sufficiente ma l'outline è assente, viene selezionato `digital_reconstructed`. Negli altri casi il profilo è `scanned_ocr`. La sola presenza di segnalibri non autorizza quindi un parser digitale senza OCR. La detection conserva anche pagine campionate, conteggi, mediana dei caratteri, motivazioni e un punteggio di confidenza calcolato mediante regole esplicite.

Il punteggio non è una probabilità di correttezza calibrata sperimentalmente. Anche la copertura campionaria è un indicatore operativo, non una misura di completezza del documento: pagine problematiche possono trovarsi fuori dal campione. Inoltre, la detection accerta la presenza dell'outline, non la sua correttezza semantica. Il controllo delle destinazioni e l'associazione ai contenuti appartengono agli stadi successivi.

La Figura 2.2 rende esplicita la precedenza del criterio testuale rispetto alla presenza dell'outline.

PDF digitale con outline

Il profilo `digital_outline` separa l'acquisizione della struttura da quella del contenuto. `PyMuPDFOutlineAdapter` fornisce le voci dell'outline, il numero di pagine e il titolo disponibile nei metadati; `DoclingContentAdapter` converte il contenuto delle pagine richieste. La configurazione di Docling disattiva l'OCR e abilita il riconoscimento della struttura tabellare. I nodi esportati vengono ricondotti ai tipi canonici titolo, testo, immagine e tabella, conservando la provenienza di pagina e le coordinate utilizzabili.

L'adattatore crea ritagli PNG per immagini e tabelle a partire dal PDF e dalle bounding box. Il fattore di scala predefinito è 2. Per le tabelle acquisisce anche una rappresentazione testuale strutturata quando disponibile. La normalizzazione richiede contenuti tracciabili: un risultato privo di elementi utilizzabili non viene trattato come estrazione riuscita. L'outline fornisce invece le destinazioni iniziali dei capitoli, non direttamente l'appartenenza di ogni elemento; questa viene calcolata dall'assemblatore comune.

PDF digitale senza outline

Il profilo `digital_reconstructed` applica la strategia `printed_toc_body_heading_fusion_v1`, definita nei moduli `structure/reconstruct.py`, `structure/page_map.py` e `structure/pdf_source.py`. La procedura combina indice stampato, numerazione delle pagine e titoli nel corpo del documento. Non si limita a classificare come titoli le righe con font più grande.

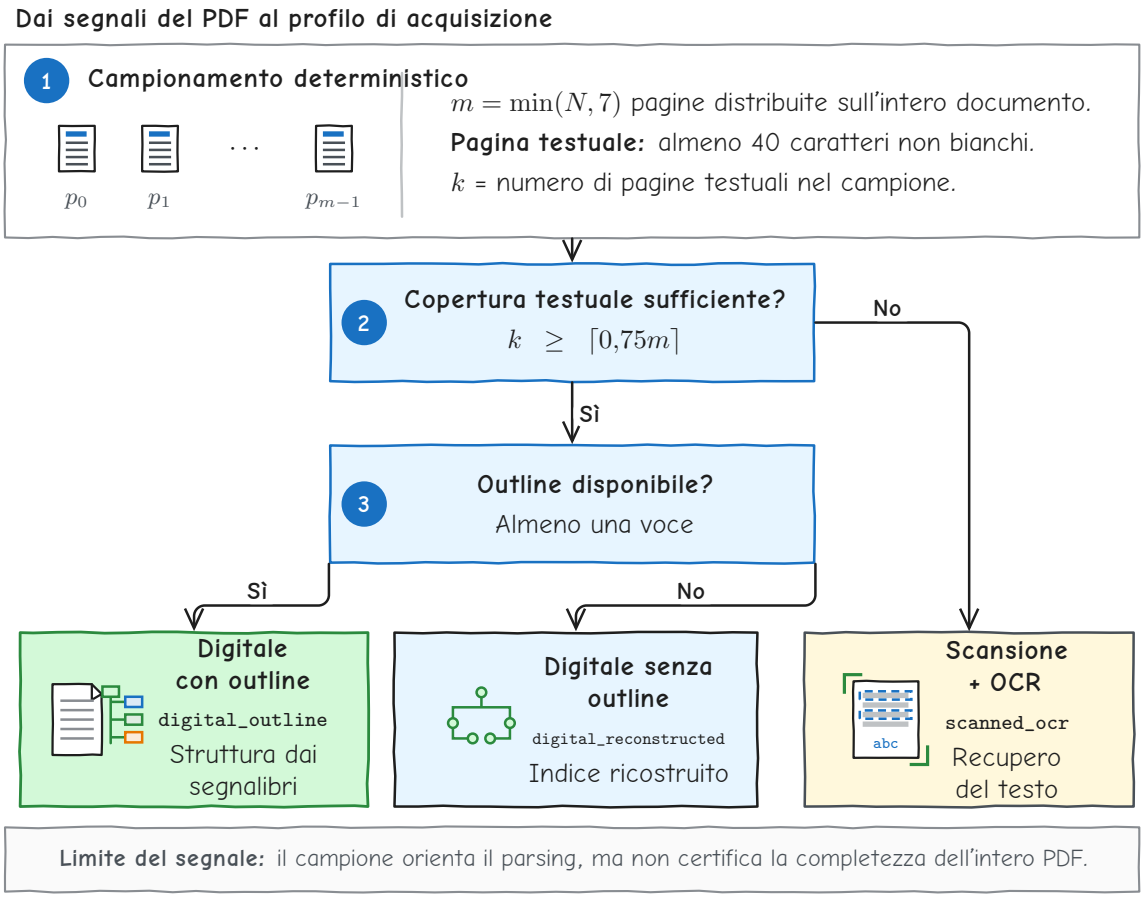


Figura 2.2: Regole di selezione del profilo dopo i controlli preliminari sul file. La copertura testuale viene valutata prima dell'outline: se è insufficiente, viene scelto il percorso OCR indipendentemente dalla presenza di segnalibri.

In primo luogo vengono raccolti segnali testuali e geometrici e costruita una mappa tra riferimenti di pagina stampati e pagine fisiche del PDF. La mappa utilizza le etichette del PDF e le evidenze di numerazione, distinguendo prefissi e disallineamenti. La ricerca dell'indice considera la porzione iniziale fino al massimo tra 80 pagine e il 25% del documento, entro i limiti del PDF, e le ultime 40 pagine. Le pagine candidate ricevono un punteggio basato su parole chiave, riferimenti terminali di pagina, sequenze di punti e marcatori di numerazione; viene scelto un blocco contiguo di indice.

Dopo aver eliminato righe accessorie ripetute, la procedura interpreta le voci dell'indice e rileva candidati titolo nel corpo del testo. Le informazioni vengono fuse mediante normalizzazione dei titoli, somiglianza testuale e compatibilità delle pagine. La configurazione predefinita distingue una soglia di corrispondenza forte pari a 0,72 e una debole pari a 0,55, con profondità massima pari a sei. Configurazione, mappa delle pagine, righe stampate, candidati, corrispondenze e avvisi vengono conservati nella diagnostica strutturale.

L'assenza di un indice stampato riconoscibile, di righe utilizzabili o di destinazioni risolvibili determina un errore: nel percorso digitale non è previsto un ripiego generale basato soltanto sui titoli del corpo. Una volta ottenuta la TOC, un adattatore statico la presenta allo stesso assemblatore utilizzato dal profilo con outline; il contenuto resta acquisito tramite Docling. La convergenza non richiede quindi un secondo modello documentale.

PDF con acquisizione OCR

Il profilo `scanned_ocr` impiega `PaddleOCRContentAdapter`, che comunica con un processo Python separato mediante `paddle_worker.py`. L'isolamento riguarda il runtime OCR e le sue dipendenze, non un servizio generativo remoto. La configurazione del rendering prevede 200 dpi e un limite di 2600 pixel per il lato maggiore. Il budget di timeout del processo è calcolato a partire da 600 secondi per pagina, con un tetto di 7200 secondi per invocazione; la configurazione predefinita prevede gruppi di sei pagine per worker. Il parser è identificato come `paddleocr_vl`.

Il setup dichiarato usa la pipeline PaddleOCR-VL `v1.6`, con classificazione dell'orientamento, analisi del layout e OCR dei blocchi immagine abilitati; il raddrizzamento delle deformazioni del documento e il riconoscimento specifico di grafici e sigilli sono disabilitati. Sono previste operazioni di ristrutturazione delle pagine, fusione delle tabelle e riassegnazione dei livelli dei titoli. Gli elementi vengono normalizzati nel contratto comune e i ritagli sono ricondotti al rendering sorgente, tenendo conto delle trasformazioni di orientamento. Le versioni di pacchetto accettate dal codice sono PaddleOCR `3.7.0`, PaddlePaddle `3.2.1` e PaddleX `3.7.2`; le stesse identità sono registrate nella diagnostica della run V4 scansionata. In quella selezione le quattro pagine sono elaborate in un solo batch, mentre la matrice storica documenta sei batch per il manuale completo.

Dopo l'acquisizione OCR si distinguono due condizioni. Se il PDF possiede un outline, questo viene riutilizzato con la strategia `embedded_outline_with_paddle_ocr`. In caso contrario, `structure/ocr_reconstruct.py` ricostruisce la TOC dagli elementi canonici: riconosce righe di indice in testi e tabelle, individua titoli strutturali e cerca corrispondenze tra numerazione stampata e pagine fisiche. L'inferenza dello scarto di numerazione richiede almeno due evidenze; la soglia di somiglianza dei titoli è 0,72. Se viene rilevato un indice stampato, almeno l'80% delle righe deve essere risolto, altrimenti la ricostruzione fallisce.

Solo in assenza di un indice stampato è previsto il ripiego sui titoli strutturali OCR, identificato come `ocr_structural_titles_fallback_v1`. Questo percorso ha confidenza predefinita inferiore e non soddisfa la politica di promozione ordinaria. Se mancano anche titoli utilizzabili, viene sollevato un errore. Le pagine riconosciute come indice stampato sono conservate nel documento, ma i relativi elementi vengono spostati alla radice ed esclusi dall'uso RAG previsto: l'indice resta così evidenza della ricostruzione senza essere confuso con il contenuto dei capitoli.

2.4 Modello documentale canonico e assemblaggio gerarchico

I contratti in `models.py` utilizzano Pydantic e dichiarano la versione di schema `1.1`. I modelli vietano campi aggiuntivi non previsti e applicano vincoli su pagine, coordinate, percorsi e relazioni tra attributi. Il `ManualDocument` contiene identificativo, titolo, nome del sorgente, lingua, metadati di elaborazione e una lista di contenuti. Quest'ultima ammette sia capitoli ricorsivi sia elementi direttamente alla radice.

La Tabella 2.2 riassume le responsabilità dei componenti.

La Figura 2.3 esplicita le relazioni di contenimento e le molteplicità dei contratti. Integra la Tabella 2.2: distingue gli elementi direttamente alla radice da quelli nei capitoli e le informazioni obbligatorie da quelle opzionali.

I tipi di elemento sono `title`, `text`, `image` e `table`. Le immagini richiedono un percorso all'asset; le tabelle richiedono almeno un ritaglio o una rappresentazione nel campo `table_markdown`. La provenienza distingue la pagina fisica del PDF dalla numerazione stampata usata nella ricostruzione dell'indice. Le bounding box devono avere area positiva e, quando sono note le dimensioni della pagina, rimanere entro tali limiti. Questi vincoli verificano la consistenza geometrica, non che il rettangolo individui l'oggetto corretto.

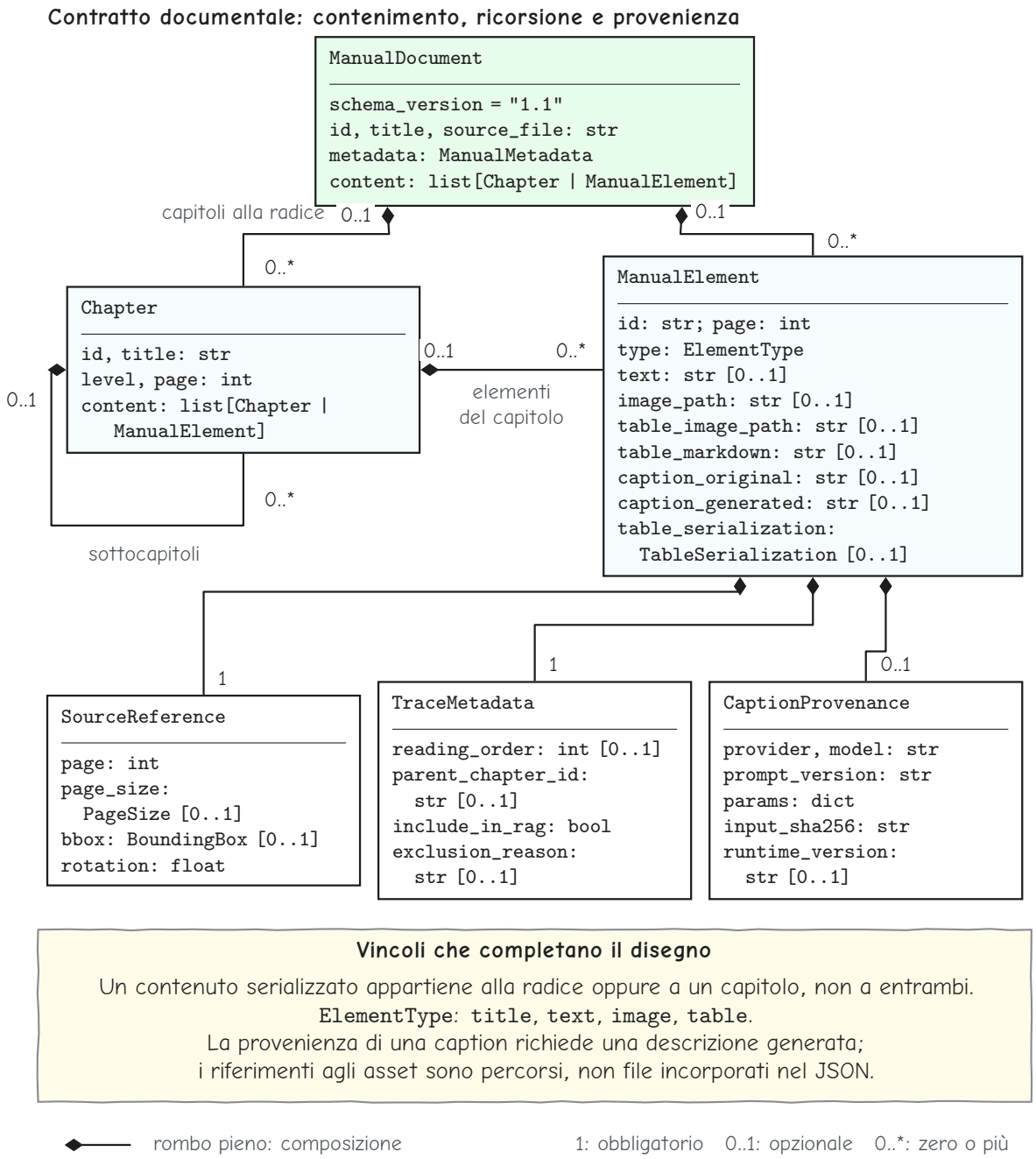


Figura 2.3: Diagramma UML semplificato delle classi del modello canonico, ricavato da `models.py`. Le composizioni descrivono il contenimento nella rappresentazione serializzata, non la gestione degli oggetti in memoria. Sono omessi metodi, ereditarietà da Pydantic e alcuni attributi e classi accessorie. Le molteplicità rendono espliciti contenuti ricorsivi e informazioni opzionali; non certificano la correttezza semantica delle associazioni.

Tabella 2.2: Componenti principali della rappresentazione canonica. Le informazioni opzionali non sono necessariamente presenti per ogni elemento.

Componente	Informazioni e funzione
`ManualMetadata`	Pagine totali ed elaborate, profilo, parser, strategia strutturale e disponibilità della TOC.
`Chapter`	Identificativo, titolo, livello, pagina iniziale e contenuti, inclusi i sottocapitoli.
`ManualElement`	Tipo, pagina e contenuto; riferimenti ad asset, caption, serializzazione tabellare e tracce.
`SourceReference`	Pagina sorgente, dimensioni della pagina, bounding box e rotazione, ove disponibili.
`TraceMetadata`	Ordine e collocazione; ruolo e livello del titolo, inclusione prevista, esclusione e continuazione.
`CaptionProvenance`	Provider, modello, versione del prompt, parametri, hash dell'ingresso e versione del runtime.

L'assemblaggio condiviso è implementato in `profiles/digital_outline.py` e riutilizzato dagli altri due profili mediante adattatori della struttura. Da ogni voce TOC viene costruito un capitolo. Una pila dei livelli determina i rapporti gerarchici; finestre di pagina delimitano la porzione associabile a ogni voce, tenendo conto della successiva voce di livello uguale o superiore. Gli elementi acquisiti vengono ordinati e ricevono il riferimento al capitolo di destinazione. Gli elementi precedenti alla prima destinazione restano alla radice, accompagnati da un avviso, invece di essere eliminati.

L'associazione basata soltanto sulla pagina è insufficiente quando più sezioni iniziano sulla stessa pagina. In questo caso l'assemblatore cerca ancoraggi negli elementi di tipo titolo e ne usa l'ordine di lettura per stabilire i confini. Alla corrispondenza esatta tra titoli normalizzati affianca confronti approssimati, anche su titoli distribuiti in più elementi, e il riconoscimento di numerazioni di sezione univoche. Per la corrispondenza approssimata testuale sono richiesti un punteggio almeno pari a 0,90 e un margine di almeno 0,04 sul secondo candidato. Ambiguità, ancoraggi mancanti e conflitti di ordine producono avvisi espliciti.

La distinzione tra TOC iniziale e documento assemblato è sostanziale: la prima descrive titoli, livelli e destinazioni; il secondo associa a tale struttura i contenuti effettivamente acquisiti. Conservare entrambi consente di verificare l'allineamento fra gerarchia dichiarata e organizzazione finale. Tuttavia, identificativi coerenti e relazioni padre–figlio valide non garantiscono che un testo o una figura appartengano semanticamente alla sezione scelta.

La Figura 2.4 illustra il caso di due sezioni sulla stessa pagina e i collegamenti che rendono verificabile un elemento nella rappresentazione comune.

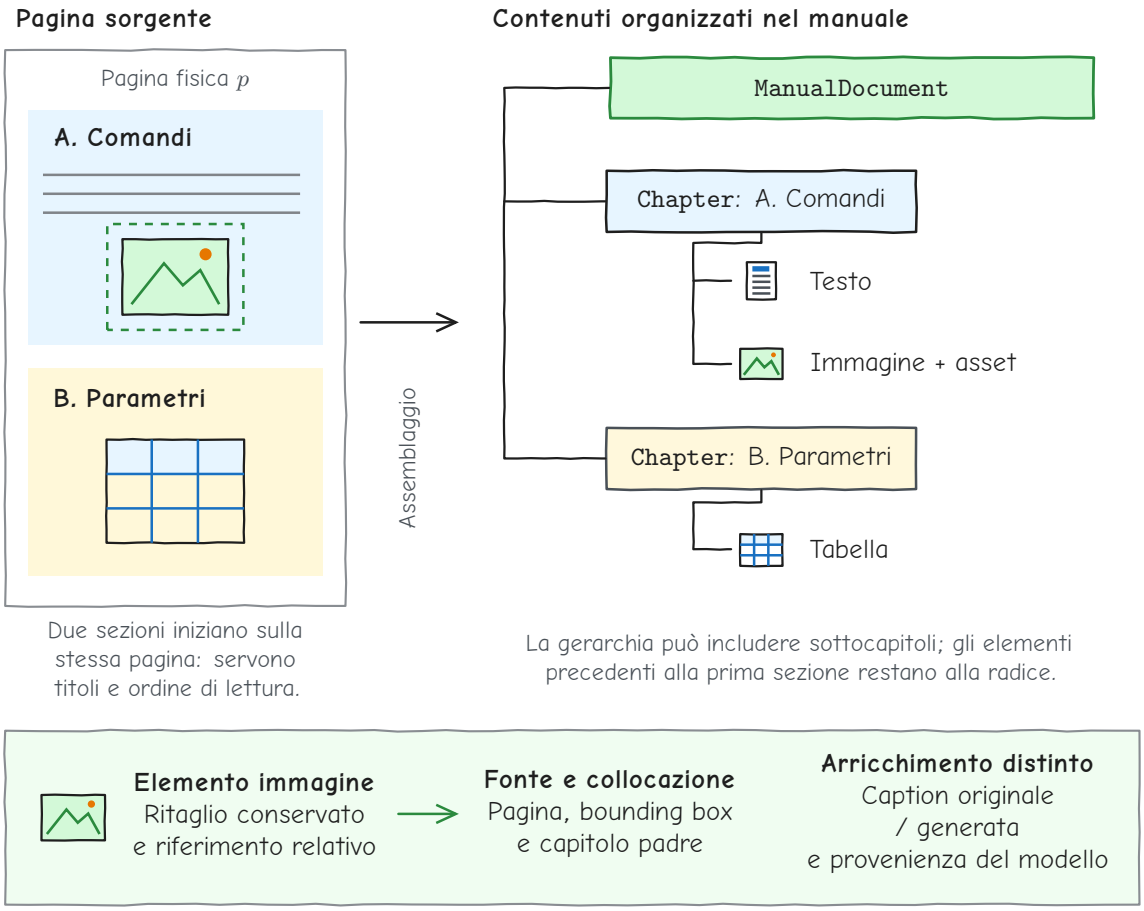


Figura 2.4: Esempio illustrativo di assemblaggio e tracciabilità, non estratto da uno dei manuali valutati. Sono rappresentati solo alcuni elementi figli. I titoli delimitano sezioni diverse anche sulla stessa pagina; il ritaglio rimane collegato alla fonte e distinto dalla descrizione generata. I dati geometrici e di provenienza sono disponibili secondo il percorso di acquisizione e l'arricchimento eseguito.

2.5 Gestione degli asset e arricchimento multimodale

Conservazione degli asset e filtro geometrico

Il bundle prevede le sottodirectory `assets/pages`, `assets/images` e `assets/tables`; il loro contenuto dipende dal parser e dagli elementi acquisiti. I riferimenti salvati nel documento sono relativi al bundle. Le immagini derivate dal PDF o dal rendering OCR restano distinte dal testo generato: una caption non sostituisce il ritaglio necessario a controllarne la correttezza.

Il filtro comune in `postprocess.py` considera immagini e tabelle e calcola, quando geometria e dimensioni della pagina sono disponibili,

$$\rho = \frac{(x_1 - x_0)(y_1 - y_0)}{W\,H}, \tag{2.2}$$

dove il numeratore rappresenta l'area della bounding box e W, H le dimensioni della pagina nello stesso sistema di coordinate. Se $\rho < 0{,}02$, l'elemento viene escluso dall'uso successivo previsto. Non viene però cancellato: sono mantenuti asset e metadati, con una motivazione esplicita. Le esclusioni preesistenti sono preservate; in assenza dei dati geometrici il filtro registra un esito non valutabile anziché assumere che il ritaglio sia piccolo.

La soglia del 2% deriva dall'ispezione del demo set descritta nel diario di progetto: in quel campione le figure complete occupavano almeno il 12% della pagina e i frammenti al massimo

l'1,3%; la soglia fu collocata nell'intervallo fra i due gruppi.[2] Si tratta di una motivazione empirica locale, non di una calibrazione su un campione indipendente né di un valore ottimale universale. Il limite inferiore del 12% descrive quel demo set, non tutte le immagini delle selezioni successive. Un simbolo di sicurezza o un componente piccolo può comunque essere informativo: la conservazione degli elementi esclusi permette di controllare il costo informativo del filtro.

Captioning locale delle immagini

`enrichment.py` definisce il contratto `CaptionProvider` e separa la logica di arricchimento dalla chiamata al modello. Il modulo restituisce una copia arricchita del documento, accompagnata dall'esito di ogni elemento visuale. Solo le immagini ammissibili sono inviate al provider; le tabelle vengono saltate perché utilizzano una serializzazione strutturata. Una caption già presente non viene sovrascritta per impostazione predefinita.

Il provider in `providers/ollama.py` utilizza Ollama con il modello `qwen3.5:4b`. Il controllo preliminare verifica il tag esatto, il digest SHA-256 atteso e la versione del servizio. Le due run V4 registrano Ollama `0.34.0`, il prompt `technical-caption-v3` e il digest del modello con prefisso `2a654d98e6fb`. I parametri effettivamente salvati sono temperatura 0, limite di generazione di 512 token, contesto di 4096 token, penalità di ripetizione 1,15 e thinking disattivato; il timeout delle richieste è di 180 secondi. Le evidenze HTTP documentano l'uso del servizio sull'interfaccia locale. Sono informazioni sulla configurazione eseguita, non garanzie di determinismo assoluto o di accuratezza.[3]

La richiesta contiene il ritaglio, il titolo del manuale, la pagina, la caption originale e un contesto testuale limitato. Nella versione corrente, il contesto privilegia i riferimenti espliciti alla figura individuati sulla stessa pagina; in loro assenza usa elementi sorgente vicini appartenenti allo stesso percorso di capitolo. Sono considerati al massimo sei elementi contestuali e il blocco risultante è limitato a 2400 caratteri nel prompt. Le caption generate non alimentano il contesto delle richieste successive. Se manca testo locale di supporto, anche il percorso strutturale viene omesso, per non imporre al modello una sezione potenzialmente errata dopo la ricostruzione OCR.

La Figura 2.5 distingue la selezione delle fonti dalla descrizione generata e mostra quali informazioni entrano nella richiesta.

Il prompt richiede sette campi: *Tipo tecnico*, *Oggetto*, *Elementi visibili*, *Relazioni/funzione*, *Valori/avvertenze*, *Rilevanza RAG* e *Incertezze*. La descrizione deve rimanere entro 1200 caratteri e l'elenco visuale è limitato a otto elementi unici. Le istruzioni impongono di basarsi sul contenuto visibile, di usare il contesto soltanto per disambiguare nomi e sigle, e di dichiarare esplicitamente letture non possibili. Il campo *Rilevanza RAG* richiede una domanda operativa breve, non una misura del recupero. Questi vincoli circoscrivono formato e lunghezza, ma il loro rispetto non prova la fedeltà alla figura. In particolare, il tetto di otto elementi può comprimere eccessivamente tavole esplose o interfacce dense, e quello di 1200 caratteri può escludere una risposta indipendentemente dalla sua qualità informativa.

La scelta finale è il risultato di iterazioni documentate, non di una ricerca di iperparametri su un insieme di test indipendente. Le prove esplorative del settembre 2026 non hanno motivato l'adozione di prompt più lunghi; V3 ha mantenuto il testo del prompt precedente e introdotto controlli aggiuntivi sulle descrizioni. Il successivo ciclo V4 ha escluso le tabelle dalle richieste al VLM, rivisto il contesto delle figure OCR e reso non bloccanti gli errori delle singole caption. Le

[2] `tesi-di-nardo/docs/decision-log.md`, voce del 13 settembre 2026 sul riesame dei punteggi e delle soglie, che ricostruisce la motivazione del filtro.

[3] Per ciascuna run in `tesi-di-nardo/enrichment-v4-final-20260915`, si vedano `evidence/provider-01-config.json`, `evidence/provider-01-observed-runtime.json` e gli scambi in `evidence/http`.

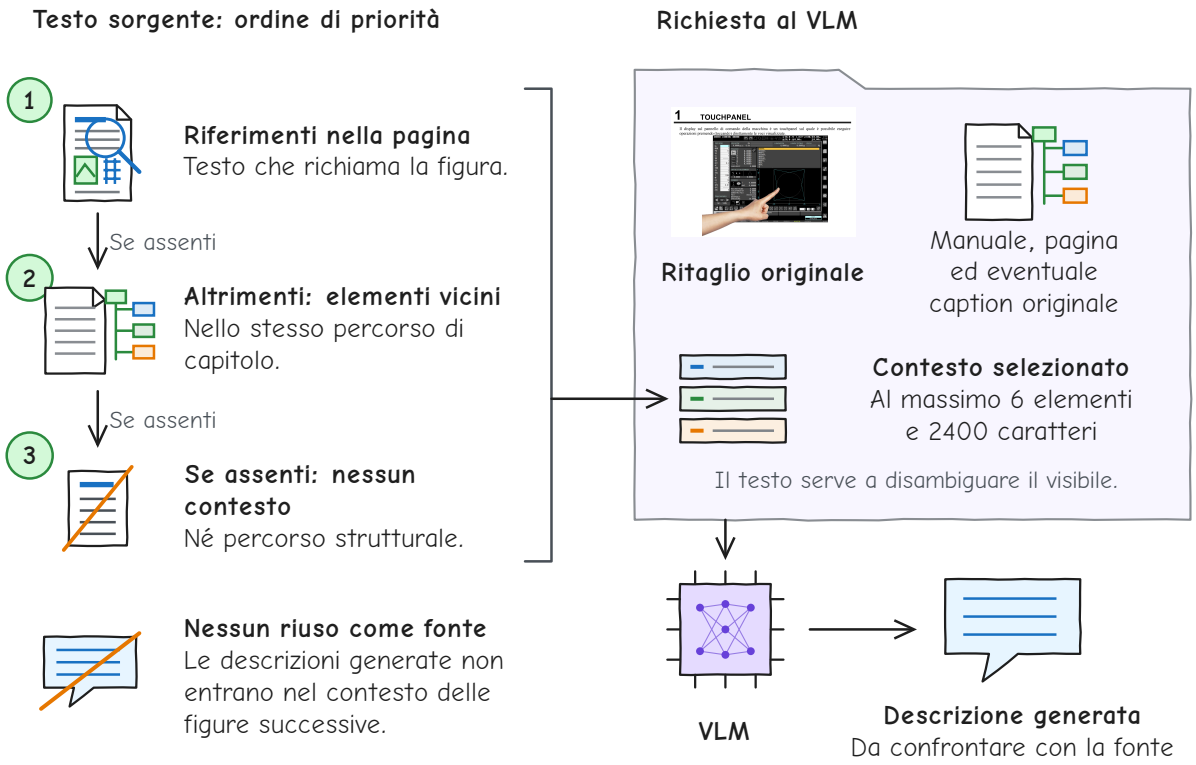


Figura 2.5: Composizione degli ingressi del captioning. Le fonti testuali seguono un ordine di priorità, non vengono sommate indiscriminatamente; in assenza di fonti ammissibili, la richiesta non contiene contesto né percorso strutturale. La miniatura dai materiali FANUC forniti è illustrativa, non un risultato del captioning. La descrizione generata va confrontata con la fonte e non diventa una nuova fonte per le richieste successive.

prove V2 con Ollama `0.31.2`, le prove intermedie su `0.33.3` e il ciclo finale su `0.34.0` devono restare distinti: l'accettazione di un setup storico non si trasferisce automaticamente a quello successivo.

Prima dei controlli, il provider normalizza gli spazi e rimuove le righe vuote della risposta: il limite di lunghezza riguarda il testo normalizzato, non il corpo HTTP grezzo. La risposta viene quindi controllata per formato e lunghezza; alcuni casi richiedono inoltre revisione. Le descrizioni di schemi elettrici e quelle con determinate contraddizioni tra letture incerte e dichiarazioni di assenza di incertezza ricevono lo stato `review_required`. La caption viene conservata, ma l'elemento è escluso dall'uso RAG previsto. Un errore di generazione o validazione produce invece `failed` per l'elemento e una motivazione di esclusione. Gli altri esiti sono `enriched` e `skipped`. Il fallimento di una singola caption non coincide necessariamente con l'interruzione dell'intera run.

La provenienza registra provider, modello, versione del prompt, parametri e versione del runtime, oltre a un hash costruito dalla richiesta e dall'asset. È quindi possibile controllare se la descrizione è ancora coerente con gli ingressi documentati. L'hash non certifica che le affermazioni generate siano vere: identifica l'ingresso, non la correttezza semantica dell'uscita.

Serializzazione deterministica delle tabelle

Il modulo `table_serialization.py` applica la strategia `header_value_rows_v1`. Interpreta il contenuto del campo `table_markdown`, che può includere rappresentazioni Markdown o HTML, e produce righe testuali con associazioni intestazione–valore. Alla riga vengono aggiunti riferimenti contestuali al manuale, al percorso di capitolo, alla pagina e alla tabella. Non vengono creati chunk né embedding: si prepara una rappresentazione tabellare utilizzabile da componenti successive.

Lo stato `structured` indica che sono state ottenute righe strutturate; `fallback` conserva in un'unica riga il contenuto normalizzato quando la struttura non è interpretabile; `unavailable` indica l'assenza di contenuto tabellare testuale utilizzabile. Quest'ultimo caso può verificarsi anche quando esiste un ritaglio della tabella. La serializzazione non può recuperare valori che il parser non ha acquisito e non usa il VLM per completarli. Mantenere separati questi stati evita di equiparare una tabella trascritta con relazioni esplicite a una semplice conservazione del testo grezzo.

La Figura 2.6 mette a confronto il percorso generativo delle immagini e quello deterministico delle tabelle, mantenendo distinti i rispettivi esiti.

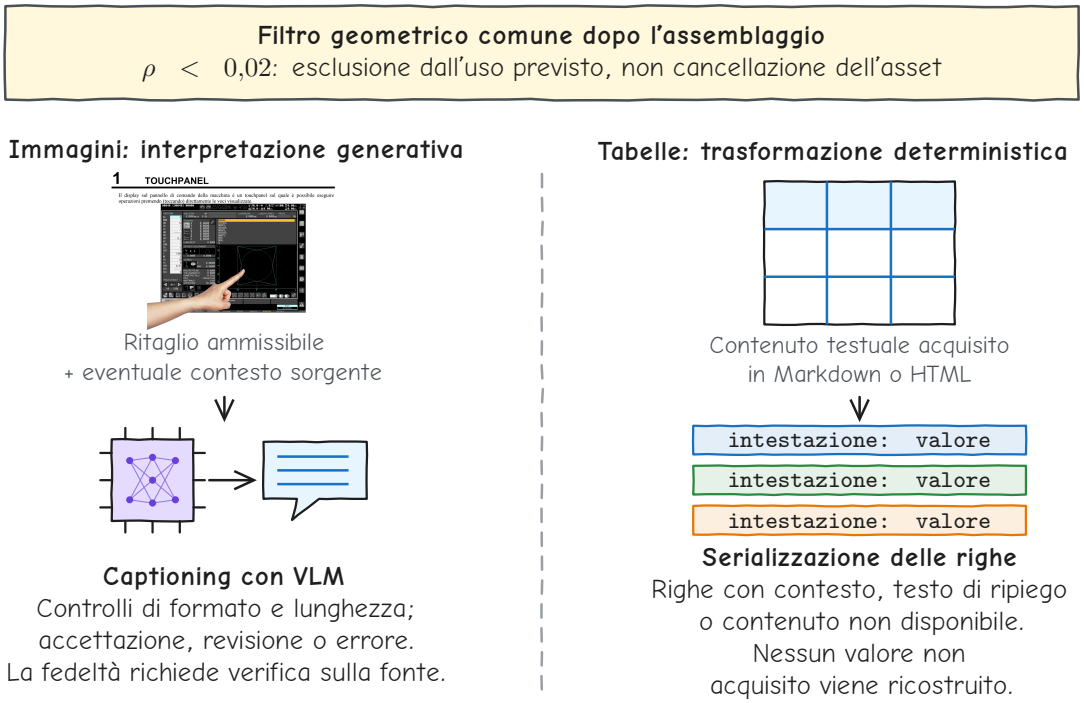


Figura 2.6: Trattamento delle immagini e delle tabelle nel workflow corrente; le colonne non indicano esecuzione parallela. Schermata del manuale FANUC dai materiali forniti come esempio visuale, non risultato del captioning; tabella e righe sono schematiche. Le tabelle non vengono inviate al VLM e sono serializzate anche se escluse dall'uso previsto, senza annullare tale esclusione.

2.6 Output, tracciabilità e controlli della pipeline

Artefatti e pubblicazione atomica

La Tabella 2.3 distingue il documento canonico dagli asset, dai metadati e dalle evidenze di controllo conservati nel bundle.

`RunBundleWorkspace`, definito in `run_bundle.py`, mantiene gli artefatti in una directory di staging sullo stesso filesystem della destinazione. La pubblicazione avviene tramite rinomina dopo il completamento dei controlli. Una destinazione già esistente viene rifiutata, così da non sovrascrivere una run precedente. In caso di errore prima della pubblicazione, lo staging viene rimosso: un insieme incompleto di file non viene presentato come bundle finale.

Il sorgente è identificato mediante nome, dimensione e SHA-256; l'orchestratore verifica che non cambi durante l'elaborazione. I percorsi degli artefatti devono rimanere relativi e non attraversare directory superiori. La diagnostica comune comprende `detection.json`, `structure.json`, `crop_filter.json`, `enrichment.json`, `runtime.json` e `promotion.json`. Il manifest e la dia-

Tabella 2.3: Artefatti del run bundle nella configurazione ordinaria.

Artefatto	Contenuto
`manual.json`	Documento canonico: gerarchia, elementi, riferimenti agli asset, descrizioni e tracce.
`toc.json`	Lista delle voci di indice con titolo, livello, pagina ed eventuale confidenza. Non è un indice RAG.
`run.json`	Manifest: identità e stato della run, sorgente e hash, configurazione, capacità rilevate, percorsi, tempi, avvisi ed errori.
`validation.json`	Esito dei controlli, messaggi e metriche tecniche; è richiesto nei bundle finali validati o sperimentali.
`assets/`	Rendering e ritagli effettivamente prodotti dagli adattatori.
`diagnostics/`	Evidenze di detection, struttura, filtro, enrichment, runtime e promozione; ulteriori artefatti del percorso OCR.

gnostica registrano configurazioni e versioni necessarie a interpretare la run, ma la riproduzione richiede anche di conservare il PDF identificato e rendere disponibile il runtime corrispondente.

La Figura 2.7 segue il passaggio dallo staging al bundle finale e distingue un impedimento alla promozione da un errore che interrompe l'esecuzione. È una vista operativa complementare ai tre piani di verifica della Figura 2.8.

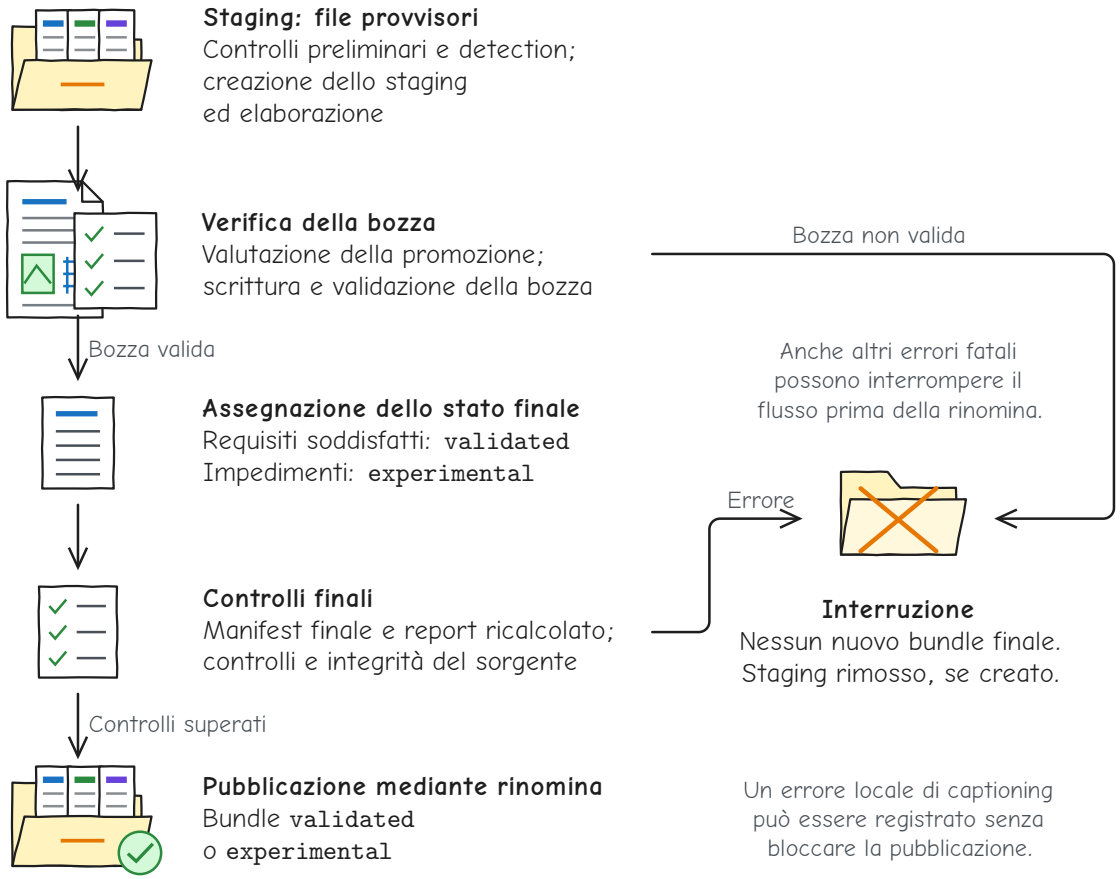


Figura 2.7: Pubblicazione secondo `orchestrator.py` e `run_bundle.py`: dal fascicolo provvisorio a quello pubblicato, oppure all'interruzione. Il segno di spunta indica la pubblicazione, non necessariamente lo stato `validated`. La conformità tecnica permette di pubblicare; la politica di promozione determina lo stato. I file nello staging sono provvisori fino alla rinomina.

Validazione tecnica e politica di promozione

Il validatore in `validation.py` ricostruisce un report a partire dagli artefatti, invece di fidarsi dell'esito già salvato. Il report dichiarato deve corrispondere a quello ricalcolato; una fingerprint del bundle contribuisce a rilevare modifiche. I controlli riguardano esistenza e schema dei file, coerenza degli identificativi e del sorgente, limiti delle pagine, unicità degli ID, relazioni padre–figlio, percorsi ed esistenza degli asset, provenienza delle caption, struttura della TOC, serializzazione tabellare ed evidenze di promozione.

Un controllo specifico impone che gli elementi canonici coprano almeno il 75% delle pagine dichiarate elaborate. Indicando con P l'insieme delle pagine elaborate e con E quello delle pagine contenenti almeno un elemento, il requisito è

$$|E \cap P| \geq \lceil 0{,}75|P| \rceil \,. \tag{2.3}$$

Questa soglia è distinta da quella usata nella detection: la prima riguarda il risultato del parsing, la seconda il campione testuale di ingresso. Nessuna delle due dimostra che ogni contenuto di una pagina sia stato acquisito, né che tutte le pagine prive di elementi siano effettivamente vuote.

La validazione controlla anche il setup software rispetto alle versioni accettate nella politica implementata. Nelle due run V4 sono registrati Python `3.12.13`, `manual_ingestion 1.0.0`, Pydantic `2.13.4`, PyMuPDF `1.28.0` e Pillow `11.3.0`; la run digitale registra inoltre Docling `2.112.0`, quella OCR le versioni Paddle indicate nella Sezione 2.3. Queste identità sono riscontrabili nei manifest e nella diagnostica, ma la versione del pacchetto non identifica da sola ogni revisione dei sorgenti. Il diario riferisce una postazione M1 Max con 32 GB per lo sviluppo; gli artefatti finali non contengono un inventario hardware sufficiente a trattare le durate registrate come un benchmark prestazionale riproducibile su quella configurazione.

La promozione è definita separatamente in `promotion.py`. Per dichiarare una run `validated` non basta che il JSON sia leggibile: occorrono copertura dell'intero documento, enrichment conforme, identità coerenti delle caption, assenza di casi che richiedono revisione, assenza di avvisi impeditivi, accettazione del profilo e struttura compatibile con la politica. Per l'OCR la politica registra un identificativo di evidenza di accettazione e distingue la ricostruzione mediante indice dal ripiego sui soli titoli. Nel caso dell'indice stampato OCR, la promozione richiede la risoluzione del 100% delle righe: la soglia dell'80% consente alla ricostruzione di proseguire, ma non basta per dichiarare la run validata.

Occorre quindi distinguere tre piani:

Conformità tecnica. Il report assume `passed` soltanto se tutti i controlli previsti sono superati.

Stato della run. Una run tecnicamente coerente è `experimental` se non soddisfa i requisiti di promozione; è `validated` quando li soddisfa. Un report `passed` è compatibile con uno stato `experimental` correttamente motivato.

Correttezza semantica. La fedeltà di testi, gerarchia, ritagli e descrizioni alla fonte richiede un confronto documentale distinto. Non è dimostrata dal nome `validated`.

I contratti definiscono anche `incomplete`, `completed` e `failed`; il percorso ordinario usa `completed` per il manifest provvisorio e pubblica l'esito finale soltanto dopo la verifica. Un errore fatale non produce normalmente un bundle finale con stato `failed`. La CLI può invece scrivere, mediante `-write-failure-report`, un report compatto accanto alla destinazione, con il contesto dell'interruzione. È quindi necessario raccogliere anche tali esiti quando si valuta la robustezza, evitando di considerare soltanto i bundle pubblicati.

La Figura 2.8 riassume la separazione tra conformità tecnica, promozione della run e fedeltà dei contenuti alla fonte. Questa distinzione orienta il metodo di valutazione descritto nel seguito.

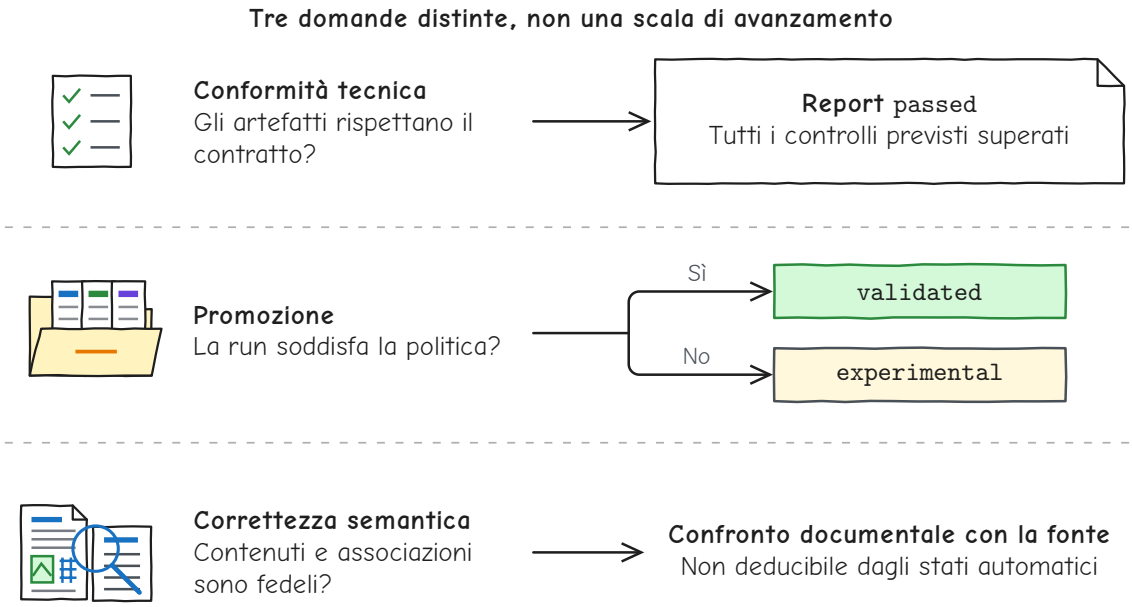


Figura 2.8: Tre domande distinte, non tre tappe equivalenti. La checklist riguarda i controlli tecnici, il fascicolo lo stato della run e la lente il confronto con la fonte. Un report `passed` è compatibile con una run `experimental`; nessuno dei due stati dimostra la correttezza semantica.

2.7 Metodo di valutazione

La valutazione combina verifica software, prove documentali per profilo e analisi descrittiva dell'enrichment. Non è un benchmark pubblico di parser né uno studio di efficacia del RAG. Le prove complete e le selezioni visuali rispondono a domande diverse: le prime verificano la produzione di una rappresentazione coerente su un intero input; le seconde rendono ispezionabili richieste, informazioni aggiunte ed errori del modello. I test, la matrice di accettazione e le registrazioni finali documentano questo disegno, mantenendo distinta l'evidenza direttamente verificabile da quella conservata soltanto nei report.

La Figura 2.9 organizza le sottosezioni seguenti in un fondamento trasversale e tre assi complementari. La sintesi evita di trattare come fasi consecutive attività che rispondono a domande diverse e usano evidenze differenti.

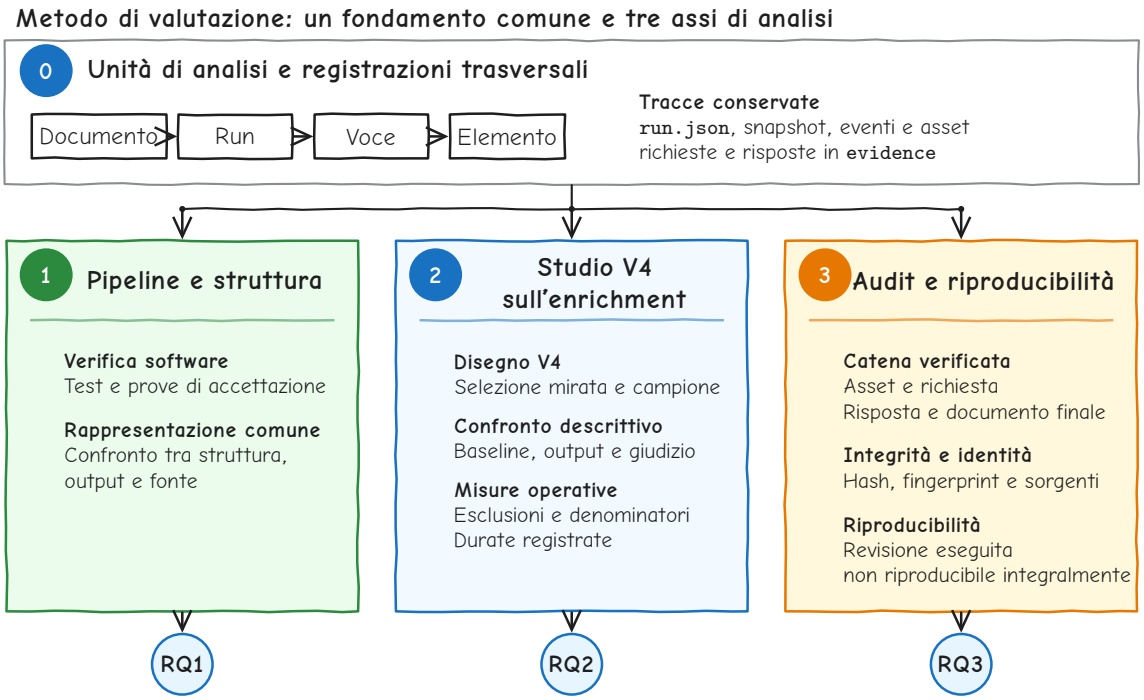


Figura 2.9: Organizzazione del metodo di valutazione. Le unità e le registrazioni sono trasversali; le altre sottosezioni confluiscono in tre assi complementari, rispettivamente collegati alla rappresentazione comune, all'enrichment V4 e alla verificabilità degli artefatti.

Unità di analisi e registrazione delle esecuzioni

Si distinguono quattro unità: documento, run, voce gerarchica ed elemento. Il documento identifica la fonte; la run identifica una particolare elaborazione e configurazione; la voce gerarchica consente di verificare titolo, livello, destinazione e relazione con il padre; l'elemento consente di controllare acquisizione, appartenenza, asset ed eventuali informazioni derivate. Ripetizioni o prove parziali sullo stesso PDF non devono essere conteggiate come documenti indipendenti.

Per le run V4 questi dati sono conservati in `run.json`, `validation.json` e nella directory `evidence`. Quest'ultima registra comando e argomenti effettivi, configurazione del provider, esito della CLI, eventi temporali, chiamate di enrichment e scambi HTTP. Le snapshot `manual-before.json` e `manual-after.json` delimitano la trasformazione generativa; l'inventario degli asset e gli hash consentono di collegare la richiesta al ritaglio effettivamente inviato. Il conteggio delle chiamate proviene dalle registrazioni, non dal solo numero di caption presenti nel documento finale.

Le esecuzioni senza provider e quelle su un sottoinsieme delle pagine restano identificate come diagnostiche. L'unità per descrivere l'esito della pipeline è il tentativo di esecuzione, inclusi eventuali fallimenti senza pubblicazione; l'unità per descrivere il captioning è la singola richiesta al provider. Gli esiti delle prove esplorative V3 non sono aggregati a quelli del ciclo V4: il comportamento sugli errori e il trattamento delle tabelle sono cambiati. L'archivio V4 documenta due tentativi, non il totale di tutte le prove svolte durante lo sviluppo.

Verifica software e prove di accettazione

La directory `tests` contiene 22 moduli di test. La loro funzione è verificare contratti e transizioni osservabili, anche con PDF sintetici, provider simulati e risposte OCR controllate; non attribuire accuratezza semantica ai motori reali. La Tabella 2.4 riassume le aree pertinenti al metodo. La matrice storica registra una suite di 329 test superati per il rilascio allora verificato: quel conteggio

non viene trasferito automaticamente allo stato dei sorgenti qui archiviati, né presentato come una nuova esecuzione della suite.

Tabella 2.4: Copertura funzionale dei test in `tesi-di-nardo/tests`, distinta dalla verifica semantica del corpus.

Area	Comportamenti verificati dai test presenti
Detection e profili	Routing, input inutilizzabili, scelta del parser e rifiuto di override incoerenti.
Struttura	Offset, indice multipagina, titoli lunghi o duplicati, limiti di pagina, ancoraggi locali e ricostruzione OCR.
Asset ed enrichment	Percorsi, filtro, fonti immutabili, esclusione delle tabelle dal VLM, provenienza e risposte non conformi.
Tabelle	Markdown/HTML, celle unite, intestazioni irregolari, fallback e testo assente; purezza, determinismo e idempotenza.
Orchestrazione e bundle	Pubblicazione atomica, protezione dalla sovrascrittura, integrità, report, promozione e fallimenti.

Due test rendono esplicite le scelte finali dell'enrichment: il primo usa un provider che genera un errore se chiamato su una tabella, verificando che la tabella lo salti; il secondo costruisce una pagina con sezioni relative a figure diverse e verifica che il prompt usi soltanto il testo associato alla figura bersaglio. Queste prove isolano il comportamento voluto, ma non stabiliscono quanto spesso il riconoscimento del contesto riesca sui PDF reali. Analogamente, i test della serializzazione verificano la trasformazione delle celle già acquisite, non la correttezza dell'OCR che le ha prodotte.

Verifica della rappresentazione comune

Per il profilo digitale ricostruito il riferimento del confronto è l'outline del PDF gemello. La matrice riporta un confronto su titoli, pagine, ordine e livelli, con una correzione esplicita dell'offset di livello alla radice. Recall e precision dei titoli descrivono copertura e sovrapproduzione rispetto a quel riferimento; la corrispondenza di pagina è distinta fra esatta ed entro una pagina di scarto. Il benchmark riguarda la gerarchia, non misura automaticamente la conservazione del testo o l'associazione delle figure ai capitoli. L'harness di confronto e le corrispondenze elementari non sono inclusi nell'archivio del progetto: i punteggi restano quindi quelli delle prove storiche, senza un nuovo calcolo.

Per l'OCR, la prova integrale documentata considera l'indice stampato a pagina 5 del WM250, lo scarto tra numerazione stampata e pagina fisica e la risoluzione delle relative voci. La matrice descrive anche l'ispezione delle pagine 26 e 33, ruotate di 180 gradi, insieme ai ritagli. Si tratta di verifiche localizzate, non di una trascrizione di riferimento completa del manuale. La run V4 sulle sole pagine 8, 10, 26 e 33 non comprende l'indice a pagina 5 e usa il fallback sui titoli OCR: il suo limite strutturale non va confuso con l'esito del percorso completo mediante indice.

La verifica semantica ha come riferimento la fonte visuale, non il solo JSON. Per gli elementi si distinguono conservazione del testo, completezza del ritaglio, associazione al capitolo e ordine di lettura. Il confronto deve essere bidirezionale: partire dall'output permette di individuare contenuti errati o duplicati, ma solo partire dalla pagina sorgente consente di rilevare omissioni. Poiché l'archivio del progetto non include i PDF integrali, il confronto sui ritagli digitali riguarda le figure conservate e lascia aperta la verifica di eventuali omissioni nelle pagine originarie. Per la selezione OCR sono invece conservati anche i rendering delle quattro pagine.

Le tabelle richiedono un confronto tra celle della fonte, rappresentazione acquisita e righe serializzate. È necessario distinguere errori già introdotti dal parser da errori di serializzazione; il ricalcolo deterministico controlla la seconda trasformazione, non la trascrizione originaria.

Analogamente, per le figure bisogna separare ritagli errati, contesti errati e descrizioni errate, poiché si tratta di difetti di stadi differenti.

Le tabelle reali appartengono alle prove storiche complete e alle selezioni delle pagine 3 e 1004, non al campione visuale finale V4. Il confronto riportato nella matrice distingue struttura disponibile, fallback e contenuto testuale assente; non giustifica l'attribuzione di accuratezza cella per cella all'intero corpus. La disponibilità dei test e dei conteggi storici permette di documentare il metodo di serializzazione, senza presentare i due bundle V4, privi di elementi `table`, come una nuova prova empirica di tale componente.

Disegno e campione dello studio finale V4

Lo studio V4 adotta una selezione mirata e descrittiva di contenuti reali, non un campionamento casuale. Il diario del 15 settembre 2026 stabilisce un ciclo digitale con outline e uno scansionato, senza replicare le immagini digitali nel profilo ricostruito. Le pagine effettivamente elaborate e la popolazione visuale risultante sono ricostruibili dai comandi, dalle snapshot e dai report del filtro, come sintetizzato nella Tabella 2.5.[4]

Tabella 2.5: Formazione del campione V4: il filtro esclude immagini prima della generazione, non in seguito a errori del VLM. Le selezioni non contengono elementi canonici di tipo tabella.

Caso	Pagine PDF elaborate	Immagini	Escluse dal filtro	Chiamate
Digitale con outline	73, 93, 98, 225, 228, 352	16	7	9
Scansione WM250	8, 10, 26, 33	9	1	8
Totale	10 pagine su due documenti	25	8	17

Il caso digitale è registrato come `enrichment-v4-outline-20260915`; quello scansionato come `enrichment-v4-scanned-20260915`. Nel digitale il campione comprende uno schema di collegamento elettrico, il touch panel, la tastiera di movimento e schermate o barre di navigazione dell'interfaccia. Nello scansionato comprende fotografie di gruppi meccanici e comandi, oltre a due tavole esplose. Queste classi sono ricavabili dai ritagli; non vengono dedotte dall'etichetta *Tipo tecnico* generata dal modello, che può essere errata.

Le 17 richieste riguardano 17 asset con digest distinti. La selezione mira a rendere osservabili contenuti spaziali, sigle, comandi e relazioni fra figura e testo, senza pretese di rappresentatività statistica dei manuali. Le pagine sono fissate prima delle due esecuzioni documentate, ma provengono da un percorso di sviluppo che ha già utilizzato materiali dello stesso progetto: non costituiscono un insieme di test indipendente dalle scelte di prompt e filtro. Ogni richiesta dispone di una sola risposta nel ciclo archiviato; non sono previste repliche da cui stimare la variabilità della generazione.

Baseline e confronto descrittivo

La baseline è la rappresentazione acquisita prima dell'enrichment: testo della pagina, eventuale `caption_original`, collegamenti documentali e asset. Non è una risposta di un sistema RAG alternativo. Nelle nove immagini digitali inviate al modello la caption originale è assente, ma esiste testo contestuale nelle pagine; nelle immagini OCR possono essere già presenti sigle o

[4] I comandi e gli ingressi effettivi sono conservati nei due file `evidence/command.json`; i conteggi sono riscontrabili in `bundle/diagnostics/crop_filter.json` e nelle directory `evidence/captions`, sotto ciascun caso in `tesi-di-nardo/enrichment-v4-final-20260915`.

frammenti di testo riconosciuti. La baseline comprende quindi anche informazioni testuali sulla figura disponibili al di fuori della caption originale. La snapshot `manual-before.json` e il prompt effettivo permettono di distinguere ciò che è già disponibile da ciò che viene aggiunto dalla lettura visuale.

Il confronto descrittivo documentato nel diario affianca originale, baseline, output Qwen e giudizio breve. Nella valutazione si distinguono: aggiunte visuali supportate dalla fonte; ripetizioni di informazioni già disponibili; omissioni; dettagli o relazioni non supportati; letture errate di sigle e valori; dichiarazioni di incertezza. Non vengono introdotti punteggi di qualità aggregati né un beneficio netto presunto. Le registrazioni tecniche consentono di ricostruire i primi tre termini del confronto; le schede qualitative curate richiamate dal diario non sono comprese nell'archivio del progetto. La documentazione conservata non specifica un protocollo di annotazione indipendente con numero di revisori, accordo inter-annotatore o risoluzione dei disaccordi.

Una caption più lunga o accettata dal validatore non è, per questo, migliore. Analogamente, `review_required` è l'esito di una regola automatica che segnala la necessità di controllo, non la prova che un revisore umano sia intervenuto. Per separare i livelli di errore, l'analisi distingue ritaglio sbagliato, contesto non pertinente e interpretazione inesatta di un ingresso corretto. Un'informazione presente nel contesto ma non visibile può spiegare l'uscita del modello senza renderla una lettura fedele della figura.

Esclusioni, denominatori e misure operative

I denominatori dipendono dallo stadio. Per descrivere il filtro si considerano tutte le 25 immagini acquisite; per descrivere la generazione si considerano le 17 chiamate effettive, comprese quelle la cui risposta non entra nel documento finale. Le immagini escluse prima della chiamata, quelle fallite e quelle da rivedere devono rimanere categorie separate. La conservazione degli scarti consente di valutare anche informazione utile eventualmente sottratta all'uso successivo, anziché limitarsi ai casi accettati.

Le risposte oltre il limite di lunghezza sono conservate in `caption-response.json` e nei corpi delle risposte HTTP anche quando `caption_generated` resta vuoto nel documento. Limitare la revisione alle sole caption finali introdurrebbe quindi una selezione favorevole degli esiti. Viceversa, le caption da rivedere possono rimanere nel documento ma con `include_in_rag=false`, che ne mantiene esplicita l'esclusione dall'uso previsto.

Gli eventi e i file `call.json` distinguono durata della CLI, durata dell'enrichment e durata delle richieste al provider. Un tempo totale include anche acquisizione, rendering, controlli e registrazione degli artefatti; non può essere attribuito integralmente al modello. Le durate disponibili descrivono due esecuzioni strumentate, con diverso numero di pagine e parser: non supportano confronti di velocità fra motori né stime di throughput generalizzabili. Gli esiti operativi e gli esempi di contenuto sono destinati al Capitolo 3, mantenendo separati metodo e risultati.

Audit degli artefatti e limiti di riproducibilità

L'audit dei materiali V4 verifica la catena *asset–richiesta–risposta–documento*. Per ciascuna chiamata sono confrontati il digest del ritaglio, quello dell'immagine codificata nel corpo HTTP e il prompt salvato. Per le caption dotate di provenance si ricostruisce l'hash dall'identificativo dell'elemento, dal tipo, dal prompt e dai byte dell'asset. Si controllano inoltre gli inventari delle immagini e la corrispondenza fra snapshot dopo l'enrichment e documento finale. Poiché i due campioni non contengono tabelle, questa corrispondenza non è una prova della serializzazione tabellare.

La fingerprint del bundle viene ricalcolata su nomi relativi e contenuto di tutti i file, escludendo `validation.json`, che conserva il valore atteso. In entrambi i casi coincide con quella archiviata. I report salvati dichiarano 30 controlli superati, mentre i manifest riportano `experimental`. Questa combinazione è coerente con il significato degli stati: il ciclo è parziale e conserva limitazioni dell'enrichment, oltre al fallback strutturale nel caso OCR. Il ricalcolo verifica l'integrità del bundle rispetto all'impronta registrata; in questo controllo non vengono rieseguiti il validatore completo o l'ingestion.

Un secondo controllo riguarda `source-hashes.json`. Per ciascuna run sono registrati 34 sorgenti del pacchetto: 32 corrispondono byte per byte ai file conservati nell'archivio del progetto, mentre `enrichment.py` e `orchestrator.py` hanno digest diversi. L'inventario cita inoltre lo script sperimentale `final_run.py`, il protocollo e `pyproject.toml`, assenti dall'archivio.[5] Nella descrizione si distinguono quindi la revisione eseguita, il cui comportamento è documentato dagli artefatti della run, e l'implementazione corrente, descritta attraverso il codice e i test. La differenza tra i sorgenti limita la riproduzione esatta finché non è disponibile la revisione corrispondente, pur lasciando verificabile l'integrità dei bundle conservati.

Per ripetere integralmente lo studio occorrono i PDF identificati dagli hash, i sorgenti della revisione eseguita, lo script di strumentazione e i runtime registrati. La matrice segnala inoltre che le dipendenze transitive Python non sono congelate per hash. L'archiviazione di parametri, input e risposte permette dunque un audit sostanziale del ciclo conservato, ma non va confusa con una riproducibilità completa già dimostrata in un ambiente indipendente.

La Tabella 2.6 collega le domande di ricerca alle evidenze disponibili e ai loro limiti.

Tabella 2.6: Domande di ricerca e livelli di evidenza. Le fonti storiche e gli artefatti direttamente ispezionabili mantengono ruoli distinti.

Domanda	Evidenze utilizzate	Distinzione da mantenere
RQ1	Matrice dei tre profili, test strutturali, TOC e documenti V4.	Prove complete storiche, selezioni finali e correttezza semantica delle associazioni.
RQ2	Asset, baseline e 17 richieste/risposte V4; prove storiche e test tabellari.	Aggiunte, omissioni ed errori; generazione e serializzazione; limiti dell'analisi qualitativa.
RQ3	Manifest, provenienza, report, fingerprint e registrazioni delle chiamate.	Integrità dei bundle, promozione, identità dei sorgenti e validazione semantica.

La validità delle conclusioni resta circoscritta ai due manuali di base, alle condizioni di ingresso e alle selezioni effettivamente documentate. Un bundle strutturato e verificabile può costituire un ingresso per un sistema RAG. Gli eventuali effetti su retrieval, generazione, sicurezza operativa e carico di lavoro richiederebbero una valutazione downstream distinta, esterna ai controlli documentali svolti in questo studio.

[5] Inventari conservati in `tesi-di-nardo/enrichment-v4-final-20260915/outline/evidence/source-hashes.json` e nel corrispondente caso `scanned`; il prefisso storico `src/` è ricondotto al pacchetto `manual_ingestion` qui archiviato.

Capitolo 3

Risultati e discussione

Il capitolo presenta gli esiti della pipeline distinguendo le prove storiche riportate nella matrice di accettazione dagli artefatti conservati del ciclo V4. Le prime documentano la copertura integrale dei tre profili; sui secondi si basa l'analisi di due esecuzioni selettive, dei loro elementi e delle 17 richieste di captioning. Come introdotto nel Capitolo 2, la copertura dei manuali completi e l'accettazione delle versioni precedenti restano riferite alle prove storiche, mentre i campioni finali descrivono il nuovo setup.

Per valutare la qualità della rappresentazione, si confrontano puntualmente i ritagli, i testi acquisiti e le risposte conservate. Questa analisi è distinta dalle schede qualitative richiamate nel diario, assenti dall'archivio del progetto, e resta circoscritta agli esempi discussi, senza un'annotazione indipendente dell'intero corpus. Gli esempi rendono verificabili aggiunte informative, errori e limiti dei controlli; la valutazione resta descrittiva, senza un punteggio generale di qualità.

3.1 Esiti della pipeline integrata per profilo

Copertura integrale documentata

La matrice di accettazione riporta la produzione di una rappresentazione gerarchica per ciascuno dei tre profili. La Tabella 3.1 sintetizza le prove complete dello schema 1.0, mantenendole separate dai risultati successivi dello schema 1.1.[1]

Tabella 3.1: Esiti integrali dello schema 1.0 dalla matrice storica, non da nuove esecuzioni. Le prove senza enrichment, incluso il replay OCR, superano i 29 controlli dello schema ma non il gate dell'enrichment.

Profilo	Pagine elaborate/totali	Capitoli	Elementi	Asset
Digitale con outline	1112/1112	963	14.047	2355
Digitale ricostruito	1112/1112	958	14.047	2355
Scansione OCR	34/34	15	350	65

I due PDF digitali producono lo stesso numero di elementi e asset, ma non la stessa TOC. Questo è coerente con l'uso dello stesso parser dei contenuti e di due strategie strutturali diverse sul medesimo manuale di base. I conteggi indicano che il cambiamento del percorso strutturale

[1] `tesi-di-nardo/docs/acceptance-matrix.md`, sezione "Evidenza storica per profilo — schema 1.0". Per il digitale con outline la run è `case1-full-no-enrich-20260712c`; per quello ricostruito `case2-full-no-enrich-20260712b`; il caso OCR è documentato mediante esecuzione reale e replay delle risposte dei worker. I relativi bundle integrali non sono inclusi nell'archivio del progetto.

ha lasciato invariata la consistenza quantitativa dell'acquisizione riportata; l'identità semantica degli elementi e la correttezza delle associazioni richiedono invece un confronto dei contenuti.

Per il profilo ricostruito la matrice registra 958 voci contro i 963 segnalibri della variante originale. Riporta inoltre un confronto su titoli, pagine, ordine e livelli; tuttavia, l'assenza dell'harness e delle corrispondenze elementari non permette di ricalcolarlo. Anche i riepiloghi dei titoli corrispondenti e di quelli mancanti o extra richiederebbero il dettaglio delle regole di abbinamento per essere riconciliati. Le differenze non vengono quindi trasformate qui in una nuova stima di precision o recall.

Nel caso scansionato, la prova completa documenta un indice individuato a pagina 5, uno scarto di numerazione pari a una pagina e 15 riferimenti risolti su 15. La matrice riferisce anche l'ispezione dei ritagli delle pagine 26 e 33, acquisite con orientamento a 180 gradi. Questi risultati sostengono l'accettazione del percorso sul WM250 considerato, non su qualunque scansione o impaginazione.

Esiti delle esecuzioni del ciclo V4

Le due run V4 del 15 settembre 2026 terminano con codice di uscita zero e pubblicano un bundle. Entrambi i manifest dichiarano `experimental`, mentre entrambi i report salvati riportano `passed` con 30 controlli superati. La Tabella 3.2 riassume la rappresentazione ottenuta.[2]

Tabella 3.2: Bundle selettivi V4, entrambi sperimentali. I capitoli non rientrano negli elementi: l'outline digitale copre l'intero PDF, la struttura OCR le sole pagine selezionate. Le caption conservate includono quelle da rivedere.

Indicatore	Digitale con outline	Scansione OCR
Pagine elaborate / totali	6/1112	4/34
Pagine con elementi / elaborate	6/6	4/4
Capitoli e voci TOC disponibili	963	10
Elementi canonici	70	47
Elementi di testo	39	24
Elementi titolo	15	14
Immagini	16	9
Tabelle	0	0
Caption conservate nel documento	8	5
Controlli superati / previsti	30/30	30/30

Nel digitale la presenza di 963 capitoli non significa che tutte le sezioni siano state elaborate: l'outline è acquisito dall'intero PDF, mentre il contenuto è limitato alle sei pagine richieste. Nello scansionato la struttura deriva invece dalle sole quattro pagine selezionate. Poiché l'indice stampato a pagina 5 non è incluso, viene attivato il fallback sui titoli OCR, che produce dieci capitoli. La differenza rispetto alle quindici voci della prova completa è quindi legata anche alla selezione delle pagine e non prova, da sola, un peggioramento del motore.

Le due esecuzioni mostrano la convergenza di parser e strutture differenti sullo stesso contratto `ManualDocument`. Gli artefatti prodotti confermano la coerenza dei file; la completezza semantica dell'intero manuale resta fuori dall'obiettivo di queste due run.

[2]Dati ricavati da `bundle/run.json`, `bundle/manual.json`, `bundle/validation.json` e `evidence/outcome.json` nei casi `outline` e `scanned` di `tesi-di-nardo/enrichment-v4-final-20260915`.

Durate registrate

Le registrazioni distinguono la durata dell'esecuzione complessiva da quella dell'enrichment. Nel caso digitale sono documentati circa 77,48 secondi per la CLI e 64,79 secondi per l'enrichment; nel caso OCR circa 480,47 e 58,74 secondi, rispettivamente. I valori provengono da `outcome.json` e dal `call.json` dello stadio di enrichment, non dalla differenza approssimata tra orari visualizzati.

Il tempo esterno all'enrichment comprende operazioni diverse, tra cui preparazione, parsing, rendering e controlli. Il maggiore tempo totale della selezione OCR non può essere attribuito al captioning, né trasformato in una graduatoria di velocità fra parser: cambiano documento, numero di pagine e lavoro svolto. Si tratta di due osservazioni operative senza repliche, non di misure medie di throughput.

La Figura 3.1 separa visivamente l'enrichment dal resto del tempo CLI, mantenendo una scala comune e indicando i diversi carichi delle due selezioni.

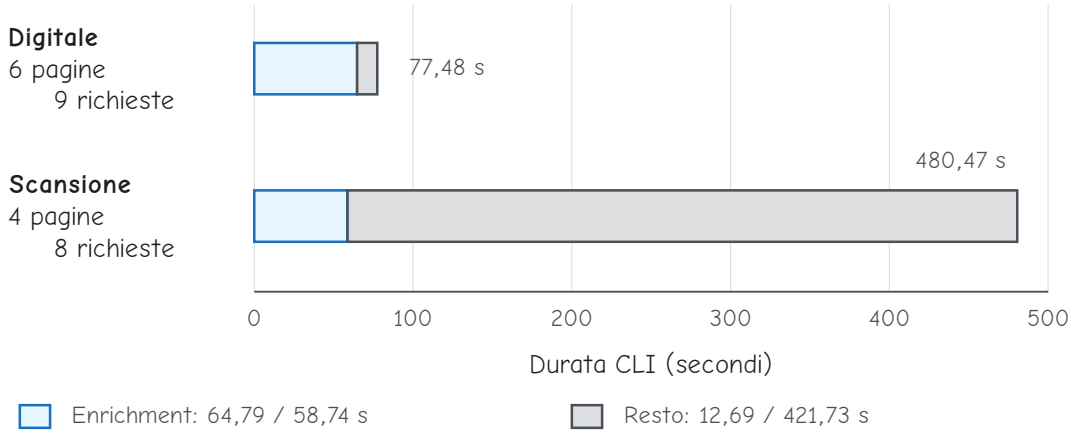


Figura 3.1: Durate delle due run V4, da `outcome.json` e dal `call.json` dell'enrichment. In legenda: digitale / scansione. Il resto è calcolato per differenza e non misura il solo OCR. Documenti e carichi diversi, senza repliche: il grafico non confronta la velocità dei parser.

3.2 Qualità della rappresentazione comune

Gerarchia coerente e associazioni non equivalenti

I report V4 non segnalano identificativi duplicati o incoerenze dei riferimenti padre–figlio. Ciò non esclude associazioni documentali semanticamente troppo deboli. Un caso concreto compare nella pagina 10 del WM250: le tre immagini della pagina sono collocate nel capitolo «8. Cross Slide Lock», mentre la prima è richiamata dai paragrafi relativi ai comandi A, B e C della Figura 11 e la seconda dai selettori D ed E della Figura 12.

Per `paddle-image-0010-0001`, il prompt effettivamente salvato usa i titoli «Emergency Button ON/OFF Switch», «Change-over Switch» e «Variable Speed Control Switch». Per `paddle-image-0010-0002` usa invece «Feed Direction Selector» e «Feed Rate Selector». Il recupero del contesto tramite richiamo alla figura seleziona dunque sezioni più pertinenti rispetto al padre canonico.[3]

Questo risultato mostra sia un recupero utile sia un limite: il contesto della richiesta viene corretto localmente, ma il riferimento gerarchico dell'immagine non viene aggiornato. Un utilizzatore del solo albero documentale riceverebbe ancora l'associazione al capitolo finale della pagina.

[3]Confronto tra `scanned/bundle/manual.json` e le richieste `scanned/evidence/captions/0004/call.json` e `scanned/evidence/captions/0005/call.json`, sotto l'archivio V4. I testi associati sono conservati anche nella snapshot precedente all'enrichment.

Non si può quindi considerare risolto il problema dell'appartenenza solo perché il prompt usa informazioni pertinenti.

La Figura 3.2 affianca il ritaglio originale alle due associazioni documentali, rendendo esplicito che il recupero del contesto non corregge automaticamente l'albero.

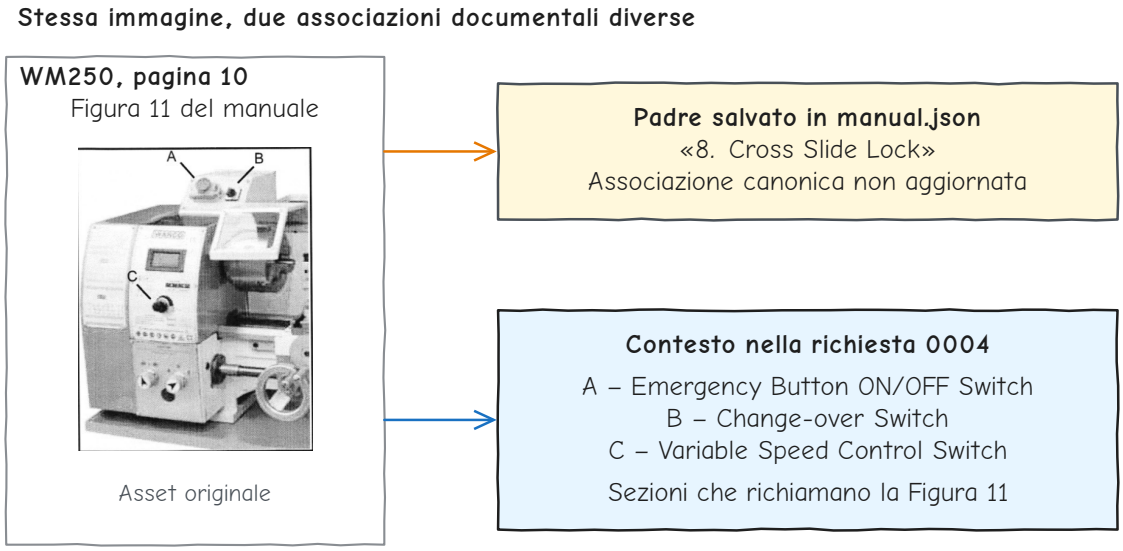


Figura 3.2: Due associazioni per `paddle-image-0010-0001`, da `manual.json` e dalla richiesta OCR `0004`. I richiami contestuali non sono nuovi capitoli. La discrepanza è ispezionabile anche se la caption viene esclusa per lunghezza.

Anche le tavole delle pagine 26 e 33 rimangono sotto lo stesso capitolo, conseguenza della struttura parziale ottenuta senza l'indice. Nelle rispettive richieste capitolo e contesto locale sono però dichiarati non disponibili: il modello non riceve come argomento il titolo di una sezione lontana e non pertinente. La precauzione riduce un possibile condizionamento della descrizione, senza ricostruire la struttura mancante.

Asset conservati ed effetti del filtro

Tutte le 25 immagini dei due campioni conservano un riferimento a un asset presente nel bundle. Il confronto fra snapshot prima e dopo l'enrichment non evidenzia modifiche al testo acquisito, ai percorsi delle immagini o alla struttura dei capitoli: le differenze riguardano caption, provenance e campi di traccia associati alle esclusioni. Questa separazione permette di riesaminare l'estrazione senza confonderla con il contenuto generato.

Il filtro geometrico esclude sette immagini digitali e una scansionata. Sei delle sette esclusioni digitali appartengono alla pagina 98, dove il parser conserva sia la tastiera complessiva sia ritagli più piccoli; la settima è a pagina 352. Gli elementi esclusi non vengono eliminati. Il filtro evita così chiamate ripetute su frammenti di una stessa figura, ma non ricompone né deduplica la rappresentazione canonica.

Il caso scansionato rende evidente il costo della regola. Il ritaglio `paddle-image-0008-0003` occupa circa l'1,76% della pagina e viene escluso perché inferiore al 2%. Come mostra la Figura 3.3, contiene tuttavia due manopole e simboli di comando riconoscibili: il filtro geometrico esclude quindi anche un asset con informazione tecnica. Non è stata generata una caption per questo asset e non è possibile attribuirgli un esito del VLM.

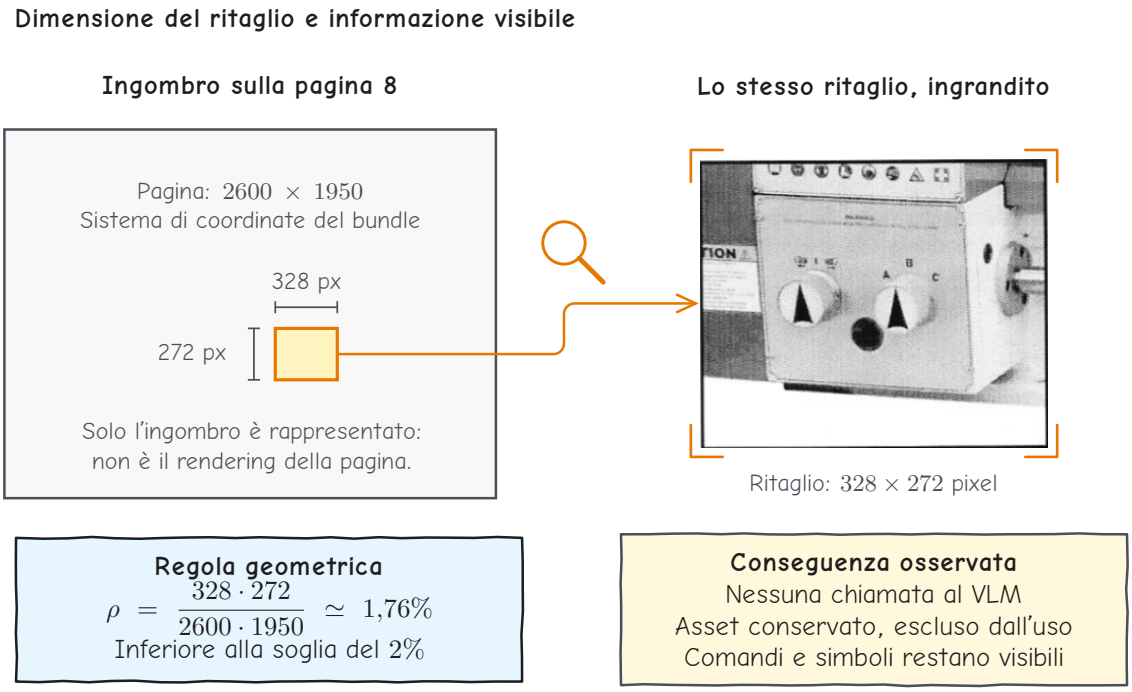


Figura 3.3: Filtro geometrico su `paddle-image-0008-0003`: a sinistra la posizione registrata in `manual.json`, a destra il ritaglio originale. L'immagine è esclusa prima della generazione, non per un errore del modello.

Il caso mostra il costo informativo della riduzione delle chiamate e l'utilità della conservazione degli scarti per valutarlo. L'effetto complessivo di un'eventuale eliminazione della soglia resta da verificare: la regola funziona come politica operativa, non come classificatore semantico validato.

Tabelle: risultati storici e limite del campione finale

Per lo schema 1.1 la matrice di accettazione riporta, in ciascuno dei due PDF digitali completi, 267 tabelle: 263 con serializzazione `structured`, quattro `unavailable` e nessuna in fallback, per 2341 righe serializzate. Le quattro indisponibilità corrispondono agli stessi elementi delle due varianti del manuale, per i quali Docling non aveva prodotto testo tabellare. Il ritaglio rimane disponibile, ma la serializzazione non ricostruisce celle assenti nell'estrazione.

Nel WM250 completo la matrice documenta dieci tabelle, nove strutturate e una indisponibile, per 289 righe. L'indisponibilità riguarda la pagina 32. Le selezioni digitali delle pagine 3 e 1004 documentano inoltre due tabelle strutturate e sei righe, inclusa la tabella degli allarmi EX1100–EX1104. Questi dati provengono dalle prove storiche, non dai due bundle V4.[4]

La distinzione fra output strutturato e testo indisponibile è un risultato utile della rappresentazione: il consumatore non riceve tutti i casi come se fossero tabelle complete. Non è invece dimostrata l'accuratezza di ogni cella o relazione intestazione–valore. Le caption tabellari riportate nelle vecchie selezioni appartengono al setup storico; nel workflow finale le tabelle sono serializzate senza essere inviate al VLM. Poiché le selezioni V4 contengono zero elementi `table`, non forniscono una nuova misura empirica di qualità tabellare.

[4] `tesi-di-nardo/docs/acceptance-matrix.md`, sezione "Chiusura dello schema 1.1" e tabella dei run con prefisso `case1-schema11`, `case2-schema11` e `case3-schema11`.

3.3 Esiti dell'arricchimento multimodale

Bilancio delle richieste e delle caption

Le 17 chiamate finali si distribuiscono in otto esiti `enriched`, cinque `review_required` e quattro `failed`. Le otto immagini saltate prima della chiamata non appartengono a questo denominatore. La Tabella 3.3 conserva la distinzione tra filtraggio, generazione e ammissibilità finale.

Tabella 3.3: Bilancio V4 da `diagnostics/enrichment.json` e `manual.json`. I gruppi descrivono stadi diversi e non vanno sommati tra loro. Tutti i 25 asset restano conservati; l'ammissibilità non misura l'accuratezza semantica.

Fase e conteggio	Digitale	Scansione	Totale
Acquisizione e filtro: 25 immagini			
Immagini acquisite	16	9	25
Escluse dal filtro prima della chiamata	7	1	8
Richieste al provider	9	8	17
Generazione: 17 richieste, tre esiti disgiunti			
`enriched`: ammissibile secondo la politica	7	1	8
`review_required`: da rivedere	1	4	5
`failed`: risposta fuori contratto	1	3	4
Conservazione nel documento finale			
Caption presenti (ammissibili + da rivedere)	8	5	13
Di cui con inclusione prevista attiva	7	1	8

Il rapporto otto su diciassette descrive le richieste che producono una caption ammissibile secondo la politica implementata. Non è una misura di accuratezza. Delle tredici caption presenti, cinque sono conservate per la revisione ma escluse dall'inclusione prevista; le altre otto sono ammissibili per l'uso successivo secondo la politica.

Aggiunte supportate e informazioni già disponibili

Nell'immagine digitale di pagina 73, `docling-image-0073-0001`, la caption nomina le fasi U, V e W con le sigle L1, L2 e L3, il riferimento QF1 e la terra di protezione PE. Queste etichette sono visibili nel ritaglio. L'output converte quindi dettagli grafici in testo esplicito, pur essendo marcato per revisione a causa della natura elettrica del contenuto.

Il guadagno informativo non coincide con l'intera descrizione: il testo di contesto già menziona cavo, interruttore QF1 e avvertenza di spegnimento. La presenza di tali informazioni nella caption non è una scoperta visuale. Il contributo osservabile è più circoscritto e riguarda la trascrizione e l'organizzazione di etichette che il contesto fornito non enumerava nello stesso modo. Il caso mostra che una descrizione esclusa per ragioni prudenziali può contenere informazioni supportate; lo stato tecnico non è un giudizio globale sul suo contenuto.

Nella seconda immagine OCR di pagina 10, `paddle-image-0010-0002`, la caption identifica i selettori D ed E e collega la figura all'avanzamento del carrello. Tuttavia, il prompt contiene già i paragrafi «Feed Direction Selector» e «Feed Rate Selector». Parte della descrizione riformula pertanto la baseline testuale. Il confronto non consente di attribuire al solo canale visuale la comprensione della funzione dei comandi, né dimostra un miglioramento del loro recupero mediante ricerca.

Errori presenti anche nelle caption accettate

Un controesempio diretto alla coincidenza fra formato e correttezza è fornito dall'immagine `docling-image-0225-0010`. La risposta descrive una barra con «10 pulsanti quadrati neri»; nel ritaglio sono visibili dodici pulsanti. La stessa risposta coglie correttamente l'evidenziazione del primo mediante un cerchio rosa, ma il conteggio è errato. La caption, lunga 486 caratteri, riceve `enriched` e l'immagine mantiene l'inclusione prevista attiva.

La Figura 3.4 permette di confrontare il conteggio visibile con l'affermazione generata e con lo stato assegnato.

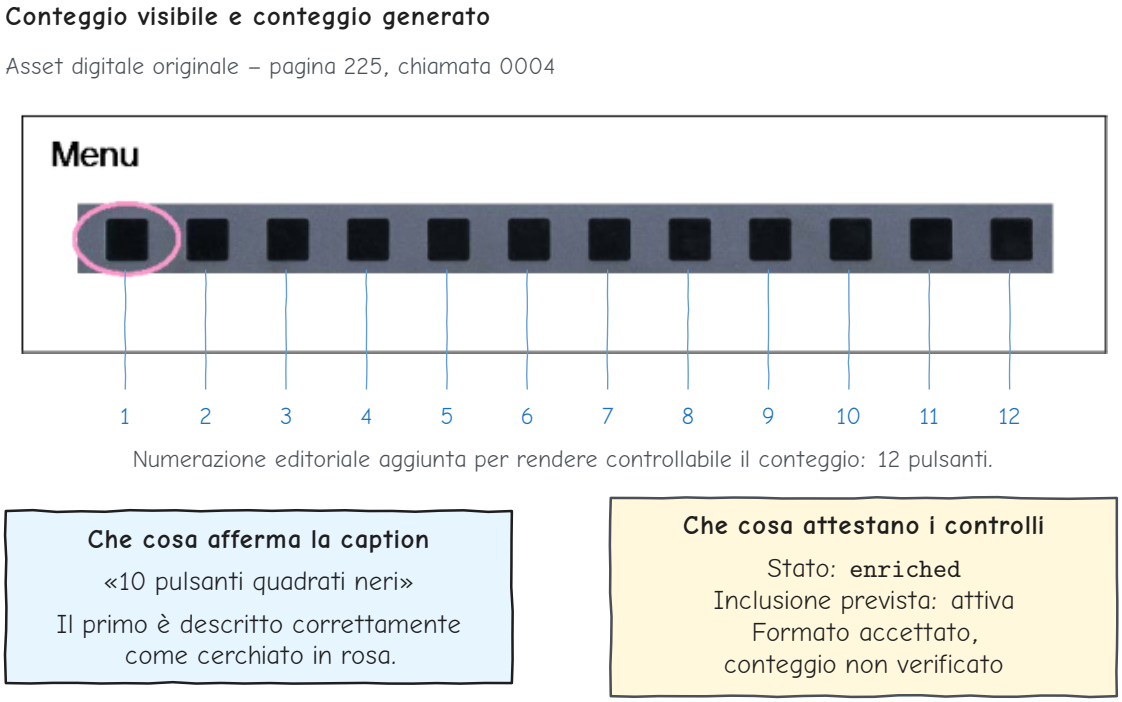


Figura 3.4: La caption di `docling-image-0225-0010` conta dieci pulsanti, ma nell'immagine sono dodici: il controllo di formato non rileva l'errore. Numeri e linee sono aggiunti per il confronto; il cerchio rosa è nell'originale.

La descrizione della schermata digitale di pagina 228, `docling-image-0228-0013`, introduce invece l'espressione «posizionamento robotico». La fonte contestuale riguarda le coordinate e i punti utente della macchina ROBOCUT e non giustifica tale qualificazione. La risposta viene esclusa per lunghezza, non perché il controllo abbia riconosciuto l'interpretazione non supportata. Il motivo tecnico del fallimento e il problema semantico osservato devono quindi essere annotati separatamente.

Questi esempi non permettono di stimare il tasso di errore dell'intero campione. Consentono però di escludere una conclusione più forte ma ingiustificata: nessuno degli stati automatici garantisce che tutti i dettagli della caption siano fedeli. Un testo formalmente corretto può includere errori insieme a osservazioni esatte.

Omissioni e comportamento dei controlli di revisione

Per la tavola esplosa OCR di pagina 26, `paddle-image-0026-0001`, la risposta si concentra su categorie generiche quali ingranaggi, cuscinetti e trasmissione. Non riproduce le associazioni fra i numerosi riferimenti numerati e le parti; una descrizione così sintetica non sostituisce la tavola quando occorre identificare un componente specifico. Il limite di otto elementi nel prompt

è compatibile con questa compressione, ma una sola esecuzione non consente di attribuirgli causalmente tutte le omissioni.

La caption classifica l'immagine come «schema/diagramma» e menziona un «motore elettrico». La diagnostica richiede revisione per schema elettrico. Il comportamento è coerente con la regola lessicale implementata, che cerca termini elettrici nei campi descrittivi di un diagramma; non dimostra però che la tavola sia uno schema di cablaggio. L'asset è una rappresentazione esplosa di un gruppo meccanico. Il caso mostra come un controllo prudenziale possa intercettare un'espressione della risposta senza classificare semanticamente il documento sorgente.

Gli altri tre casi OCR da rivedere sono `paddle-image-0008-0004`, `paddle-image-0010-0003` e `paddle-image-0033-0001`. Il motivo salvato è una contraddizione tra la dichiarazione di assenza di incertezze e una lettura indicata come non possibile. Il controllo opera su formulazioni linguistiche: non verifica autonomamente se il dettaglio sia illeggibile, assente o semplicemente non pertinente. La segnalazione è quindi un invito alla revisione e può essere conservativa; non equivale a una valutazione umana già svolta.

Nel complesso, il ciclo V4 evidenzia aggiunte supportate, ripetizioni della baseline, omissioni e interpretazioni errate. Non fornisce una misura di beneficio netto dell'enrichment. Tale conclusione non nega che alcune caption siano informative: distingue l'esistenza di contributi utili dalla dimostrazione di un miglioramento complessivo e trasferibile.

3.4 Esiti dei controlli, fallimenti e tracciabilità

Fallimenti delle caption senza perdita del bundle

Tutti i quattro esiti `failed` delle richieste V4 sono associati al superamento del limite di 1200 caratteri. Le risposte restano conservate nelle evidenze del provider, ma non vengono pubblicate come caption generate nel documento. Le lunghezze riguardano il testo normalizzato, non i token o i byte HTTP. Le chiamate escluse sono D07 (1207 caratteri; eccedenza 7), S01 (1370; eccedenza 170), S02 (1360; eccedenza 160) e S04 (1210; eccedenza 10): le sigle identificano le registrazioni digitali e OCR usate nel grafico.

La Figura 3.5 colloca le quattro esclusioni rispetto alle altre tredici risposte e alla soglia del contratto. Include anche le risposte non pubblicate come caption, evitando di rappresentare soltanto gli esiti conservati nel documento.

Le eccedenze di sette e dieci caratteri rendono evidente la natura rigida del contratto: l'esclusione non è proporzionale alla qualità del contenuto. Non ne consegue che sia corretto troncare automaticamente le risposte o aumentare la soglia sulla base di questi soli esempi. Entrambe le modifiche cambierebbero il trattamento dell'output e richiederebbero nuove verifiche, in particolare per non perdere il campo delle incertezze.

Diversamente dall'esito V3 descritto nel diario, nel quale le caption fuori contratto avevano impedito la pubblicazione, le due run V4 mantengono gli altri risultati e pubblicano bundle sperimentali. La documentazione V3 è evidenza storica, non un'esecuzione qui ripetuta; la continuità V4 è invece osservabile negli artefatti. Il miglioramento sostenuto dai dati riguarda la gestione del fallimento locale, non un aumento dimostrato dell'accuratezza delle descrizioni.

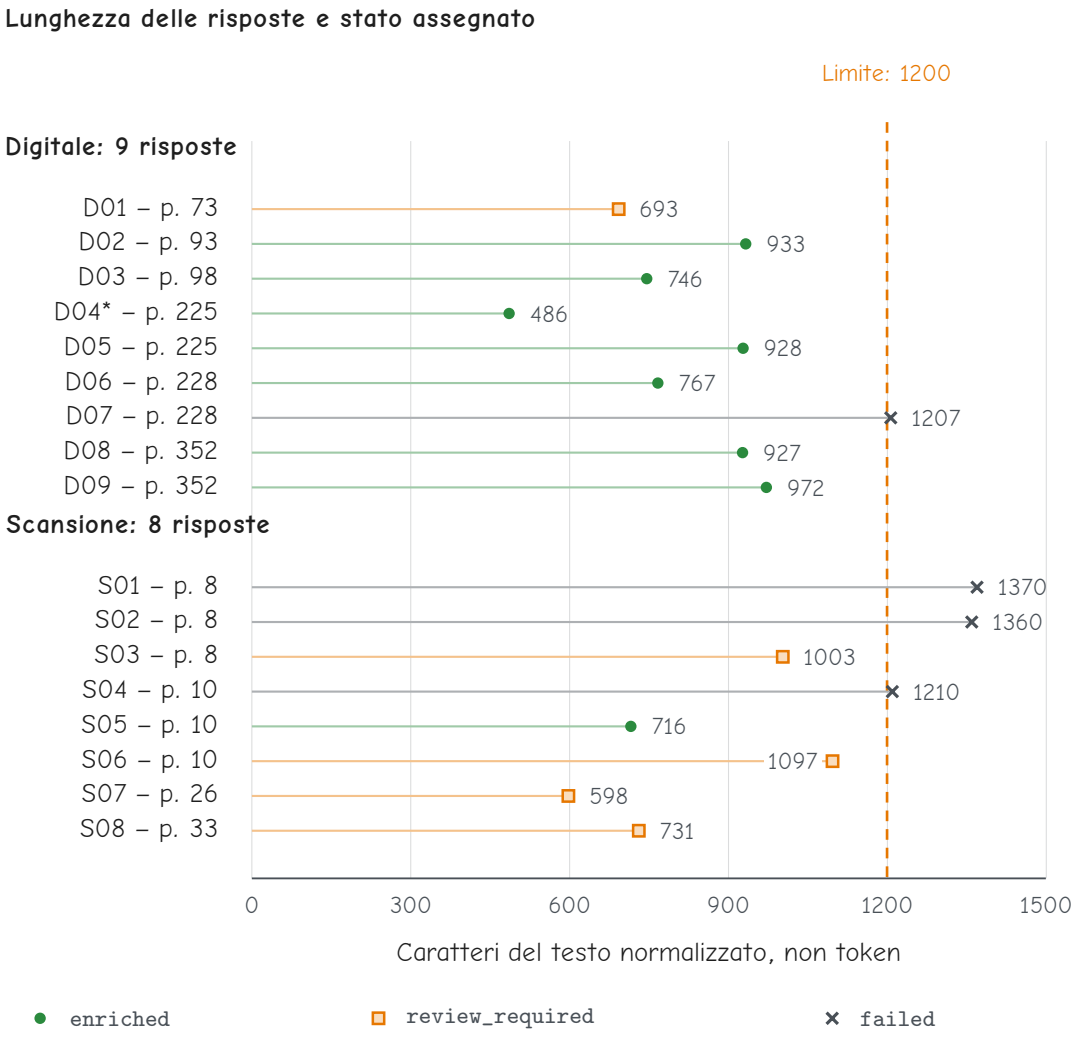


Figura 3.5: Lunghezza delle 17 risposte V4, dai `caption-response.json`, incluse le quattro escluse per lunghezza. Le otto immagini filtrate prima della generazione non compaiono. Il caso D04 è illustrato nella Figura 3.4.

Validazione e gate di promozione

I report salvati superano tutti i 30 controlli dello schema 1.1. Le ragioni che impediscono lo stato `validated` sono leggibili separatamente nella diagnostica di promozione, riportata nella Tabella 3.4.

Per entrambi i casi pesano la selezione parziale, i fallimenti del captioning e gli elementi da rivedere. Nel digitale gli avvisi su due corrispondenze normalizzate dei titoli non impediscono il superamento del gate sugli avvisi. Nel caso scansionato incidono anche i limiti della struttura parziale e il fallback. Il profilo OCR risulta accettato tramite l'evidenza storica registrata, ma questa accettazione non basta a promuovere una run che ne utilizza un percorso strutturale meno affidabile.

La combinazione fra report passato e stato sperimentale non è contraddittoria. Il report conferma che lo stato e gli artefatti rispettano il contratto e la politica; non afferma che tutti i gate di promozione siano superati. L'errore di conteggio della Figura 3.4 mostra inoltre che neppure il superamento dei controlli della singola caption coincide con la correttezza semantica.

Tabella 3.4: Gate di promozione da `diagnostics/promotion.json`. "No" indica un requisito non soddisfatto, non necessariamente incoerenza tecnica. Il requisito sull'enrichment considera anche gli errori delle singole richieste.

Requisito di promozione	Digitale	Scansione
Documento elaborato integralmente `document.full`	No	No
Enrichment configurato e senza errori `enrichment.configured`	No	No
Identità delle caption coerente `captions.identity`	Sì	Sì
Nessuna caption da rivedere `enrichment.review_free`	No	No
Nessun avviso ostativo `warnings.nonconsequential`	Sì	No
Profilo accettato `profile.accepted`	Sì	Sì
Strategia strutturale accettata `structure.accepted`	Sì	No

Integrità degli artefatti e identificazione della revisione

L'audit descritto nella Sezione 2.7 conferma la corrispondenza delle fingerprint dei due bundle con i valori salvati. I digest iniziano rispettivamente con `395963bbb6ba` e `b8827d3573df`; i valori completi rimangono nei report. Risultano coerenti anche gli hash degli asset inviati, i prompt salvati e le risposte HTTP rispetto al testo normalizzato conservato nelle evidenze. Le tredici caption presenti nel documento hanno un hash di ingresso ricostruibile dagli asset e dalle richieste.

Questa catena rende controllabili anche gli esiti esclusi: è possibile distinguere un asset mai inviato, una risposta generata ma fuori contratto e una caption conservata soltanto per revisione. I contatori del report possono essere confrontati con le chiamate individuali. L'uguaglianza fra snapshot dopo l'enrichment e documento finale, in questi campioni privi di tabelle, conferma inoltre che il contenuto generativo valutato è quello effettivamente pubblicato.

Resta una limitazione dell'identità software: 32 dei 34 sorgenti registrati per ciascuna run corrispondono ai file conservati nell'archivio del progetto, mentre `enrichment.py` e `orchestrator.py` hanno hash diversi. Mancano anche lo script di strumentazione e alcuni materiali del protocollo. L'audit conferma dunque l'integrità dei bundle rispetto alle loro impronte; per la riproduzione esatta delle run occorre invece recuperare la revisione eseguita e i materiali mancanti. La verifica si limita agli artefatti conservati, senza rieseguire la suite di test o la validazione completa del corpus.

3.5 Discussione, limiti e trasferibilità

Contributo dell'integrazione

Il risultato principale è l'organizzazione di percorsi eterogenei in un workflow con contratto comune e stati espliciti. Le prove complete documentate e i bundle selettivi mostrano che struttura, testo, asset, descrizioni e tracce possono essere rappresentati senza imporre ai consumatori il formato nativo di ciascun parser. Il valore dell'integrazione risiede anche nel rendere visibili i limiti degli stadi: una tabella può essere indisponibile, una caption può fallire e una run può restare sperimentale senza essere confusa con un output corrotto.

La convergenza formale non risolve ogni problema documentale. Il caso OCR di pagina 10 dimostra che un riferimento padre valido può essere semanticamente inadeguato. La selezione di un contesto migliore per il prompt è una mitigazione locale, non una correzione dell'intero modello gerarchico. La qualità va quindi verificata anche nelle relazioni, non soltanto nella presenza degli elementi.

Compromessi dell'enrichment e dei controlli

Gli esempi non sostengono né l'accettazione indiscriminata delle descrizioni né il loro rifiuto complessivo. Nel medesimo ciclo si osservano trascrizioni visuali supportate, informazioni già presenti nel contesto, conteggi errati e descrizioni troppo generiche. I controlli intercettano vincoli di formato e segnali linguistici, ma possono lasciare passare errori semantici o inviare a revisione casi che richiedono un'interpretazione più fine della fonte.

Anche le esclusioni hanno un costo. Il ritaglio WM250 sotto soglia contiene comandi riconoscibili; alcune risposte vengono scartate per un'eccedenza minima di lunghezza; le descrizioni elettriche restano escluse pur potendo contenere etichette corrette. La trasparenza di questi esiti consente di valutare il compromesso scelto fra copertura e prudenza, la cui ottimalità resta da verificare.

La separazione finale fra immagini al VLM e tabelle serializzate riduce le occasioni di riscrittura generativa dei dati tabellari. Non rende però automaticamente corrette le celle acquisite. La fedeltà del parser e quella della trasformazione successiva restano due problemi distinti, da valutare con riferimenti appropriati.

Validità e generalizzazione

Il corpus comprende due manuali di base e tre condizioni di ingresso; le due varianti digitali non sono indipendenti. La selezione di 17 immagini deriva dal percorso di sviluppo della pipeline, non da un campione esterno o casuale. Lo studio non comprende repliche delle generazioni o un'annotazione indipendente completa; inoltre, i PDF integrali sono assenti dall'archivio del progetto. Questi limiti impediscono di stimare generalizzazione, variabilità o tassi semantici globali con un significato statistico fondato.

Il confronto tra digitale e scansione è inoltre confuso da differenze di documento, lingua, layout e struttura disponibile. Il maggiore numero di esclusioni successive alla generazione nel caso OCR non isola l'effetto del parser o della qualità dell'immagine. Analogamente, le metriche storiche della TOC e i conteggi delle tabelle non sostituiscono l'analisi diretta delle corrispondenze o delle celle.

Infine, non sono stati misurati retrieval, qualità delle risposte dell'assistente o prestazioni degli operatori dopo l'introduzione di questa pipeline. La presenza di testo visuale aggiuntivo è una condizione che può essere sfruttata a valle, non una prova di efficacia. Le conclusioni sostenute dal lavoro riguardano l'integrazione dell'ingestion, l'ispezionabilità degli esiti e i limiti emersi nella rappresentazione e nell'enrichment.

Capitolo 4

Conclusioni

4.1 Sintesi del contributo

La tesi affronta la preparazione dei manuali tecnici prima del retrieval, distinguendo tale problema dalla costruzione dell'assistente industriale completo. Il contributo è una pipeline integrata che seleziona automaticamente una strategia di acquisizione, organizza i contenuti in una rappresentazione gerarchica comune e conserva elementi, asset e provenienza in un bundle verificabile. La CLI ne costituisce l'interfaccia operativa; i contratti, la diagnostica e la pubblicazione atomica ne rendono espliciti gli esiti.

Le evidenze combinano prove storiche documentate per tre profili e due esecuzioni selettive finali direttamente ispezionabili. I tre file di ingresso corrispondono a due manuali di base: una coppia digitale con e senza outline e un manuale scansionato distinto. Il ciclo V4 mostra l'elaborazione di 17 immagini mediante il modello locale, mantenendo separata la serializzazione deterministica delle tabelle. I bundle vengono pubblicati come sperimentali e conservano sia i risultati utilizzabili secondo la politica sia le motivazioni delle esclusioni.

La valutazione ha messo in luce limiti semantici anche in una rappresentazione formalmente coerente: un'associazione gerarchica OCR non pertinente, una caption accettata con un conteggio errato e un ritaglio informativo escluso per dimensione. Allo stesso tempo, la separazione fra fonte, descrizione e decisione rende questi limiti ricostruibili. Il contributo sostenuto dalle evidenze è quindi l'integrazione di una preparazione documentale ispezionabile, non una garanzia di accuratezza automatica né un miglioramento dimostrato del sistema RAG downstream.

4.2 Risposte alle domande di ricerca

RQ1 — Ingestion integrata

Come ricondurre manuali PDF eterogenei, attraverso strategie di acquisizione differenti, a una rappresentazione gerarchica comune, e quali limiti emergono nella conservazione dei contenuti e dei loro collegamenti?

La soluzione sviluppata separa il recupero dei contenuti e della struttura dalle trasformazioni condivise. L'outline incorporato, la ricostruzione dall'indice stampato e l'acquisizione OCR alimentano lo stesso assemblatore e lo stesso contratto `ManualDocument`. Le prove storiche riportate nella Sezione 3.1 documentano la copertura dei tre profili, mentre i bundle V4 rendono direttamente osservabile la convergenza di due percorsi su artefatti omogenei.

Il limite principale emerso è la differenza fra coerenza delle relazioni e correttezza della loro interpretazione. Nella selezione OCR più figure rimangono associate all'ultima sezione della pagina, anche quando il contesto della richiesta viene ricondotto alla figura corretta. La conservazione

degli asset permette di verificarlo, ma non lo risolve. La risposta a RQ1 è pertanto positiva quanto alla realizzazione del contratto comune e circoscritta quanto alla fedeltà: non è dimostrata la conservazione completa di ogni contenuto o relazione su famiglie di manuali diverse da quelle studiate.

RQ2 — Arricchimento

Quali informazioni aggiuntive, omissioni ed errori introduce l'arricchimento multimodale locale rispetto ai contenuti già disponibili dall'estrazione?

Il captioning può rendere esplicite etichette visuali, disposizioni e riferimenti altrimenti confinati nel ritaglio. L'esempio delle fasi e della terra di protezione mostra aggiunte supportate, mentre altri passaggi delle descrizioni ripetono informazioni già presenti nel contesto. Nel ciclo finale coesistono inoltre errori di conteggio, qualificazioni non supportate e sintesi che non preservano tutte le relazioni delle tavole tecniche, come documentato nella Sezione 3.3.

Otto delle 17 richieste ricevono lo stato `enriched`, cinque richiedono revisione e quattro falliscono il contratto di lunghezza. Questi sono esiti tecnici, non punteggi di accuratezza: la caption che indica dieci pulsanti anziché dodici rientra fra quelle accettate. Per le tabelle il workflow finale evita la generazione e conserva una serializzazione delle informazioni acquisite; i due campioni V4 non ne misurano nuovamente la qualità.

Non è quindi dimostrato un beneficio netto dell'enrichment. È invece documentata la necessità di distinguere aggiunte reali, riformulazioni della baseline, omissioni ed errori, includendo nella valutazione le risposte escluse. Una maggiore quantità di testo non può essere assunta come miglioramento della rappresentazione o della consultazione del manuale.

RQ3 — Verificabilità

In che modo provenienza e controlli consentono di verificare gli esiti della pipeline e di distinguerne la conformità tecnica dalla correttezza semantica?

Manifest, hash degli ingressi, snapshot, registrazioni delle richieste e diagnostica consentono di ricostruire quale asset è stato elaborato, con quale contesto, quale risposta è stata prodotta e perché essa sia stata inclusa, esclusa o destinata a revisione. La Sezione 3.4 mostra che un report con controlli superati può accompagnare correttamente un bundle sperimentale: la conformità riguarda anche la dichiarazione veritiera dei limiti, non soltanto il successo del percorso ordinario.

La verifica delle fingerprint e delle richieste sostiene l'integrità dei materiali conservati. Il confronto con il contenuto visuale resta però necessario per accertare le affermazioni delle caption e la pertinenza dei collegamenti documentali. Inoltre, la mancata corrispondenza di due sorgenti agli hash registrati e l'assenza di parte dei materiali limitano la riproduzione esatta. La risposta a RQ3 è dunque che la provenienza rende gli esiti controllabili e le anomalie localizzabili, ma non sostituisce né la verifica semantica né la disponibilità completa dell'ambiente sperimentale.

4.3 Sviluppi futuri

Consolidamento della pipeline documentale

La prima priorità è completare il pacchetto di riproducibilità: PDF identificati dagli hash, revisione esatta dei sorgenti, script di strumentazione, protocollo e schede qualitative. La conservazione delle dipendenze e delle caratteristiche del runtime dovrebbe permettere di ripetere le esecuzioni nelle condizioni documentate, andando oltre la verifica d'integrità dei bundle conservati. Questa attività precede l'estensione del corpus, perché rende confrontabili le prove successive.

Sul piano strutturale, il caso OCR suggerisce di collegare le figure alle sezioni che le richiamano anche nel documento canonico, non soltanto nel prompt. La correzione dovrebbe gestire riferimenti multipli e ambigui senza imporre sempre un unico padre semantico. È inoltre necessario verificare esplicitamente i documenti parziali e i confini fra sezioni sulla stessa pagina, evitando che un titolo locale venga esteso a tavole successive prive di intestazione.

Per gli asset e l'enrichment, le priorità derivano dai fallimenti osservati: riesaminare il filtro geometrico su figure piccole ma informative; distinguere meglio testo assente e testo illeggibile; verificare numeri e sigle contro la fonte; valutare controlli di revisione meno dipendenti da sole parole chiave. Le soglie di lunghezza e il limite di otto elementi richiedono una valutazione su materiali separati da quelli usati per modificarli. Allungare il prompt o ammettere tutte le caption non costituirebbe, di per sé, una soluzione ai problemi semantici.

La verifica dovrebbe estendersi a nuovi manuali, con criteri di annotazione espliciti, revisione indipendente e conservazione dei disaccordi. Repliche delle generazioni permetterebbero di distinguere errori ricorrenti e variabilità delle singole risposte. Per le tabelle occorre affiancare ai test della serializzazione il confronto tra celle, intestazioni e fonte, separando gli errori del parser da quelli introdotti dalle trasformazioni successive.

Integrazione e valutazione downstream

L'integrazione con il sistema RAG del progetto quadro costituisce una fase distinta. Un confronto controllato dovrebbe mantenere uguali corpus, chunking, modello di embedding e procedura di retrieval, variando il contributo delle caption o della rappresentazione tabellare. Le domande di prova dovrebbero includere richieste per cui figura, posizione di un comando o relazione fra celle siano effettivamente necessarie, senza limitarsi a quesiti già risolvibili dal testo circostante.

Solo dopo aver verificato il recupero e le risposte dell'assistente sarebbe possibile valutare l'effetto sull'attività degli operatori. Uno studio human-in-the-loop richiederebbe un protocollo specifico, compiti comparabili e misure di prestazione ed esperienza d'uso; non sarebbe sostituibile con i conteggi dei bundle o delle caption. Le misure del progetto quadro richiamate nell'Introduzione non sono risultati della pipeline qui studiata.

La Figura 4.1 traduce questa direzione in uno schema di confronto: la variazione documentale viene isolata prima di estendere la valutazione al lavoro degli operatori.

In conclusione, il lavoro fornisce una base documentale comune e strumenti per controllarne le trasformazioni. Il passo successivo non è assumere che l'informazione generata sia affidabile, ma verificare dove essa aggiunga contenuto corretto e dove introduca errori. È questa distinzione fra disponibilità, tracciabilità e fedeltà a delimitare il contributo della tesi e a orientarne gli sviluppi.

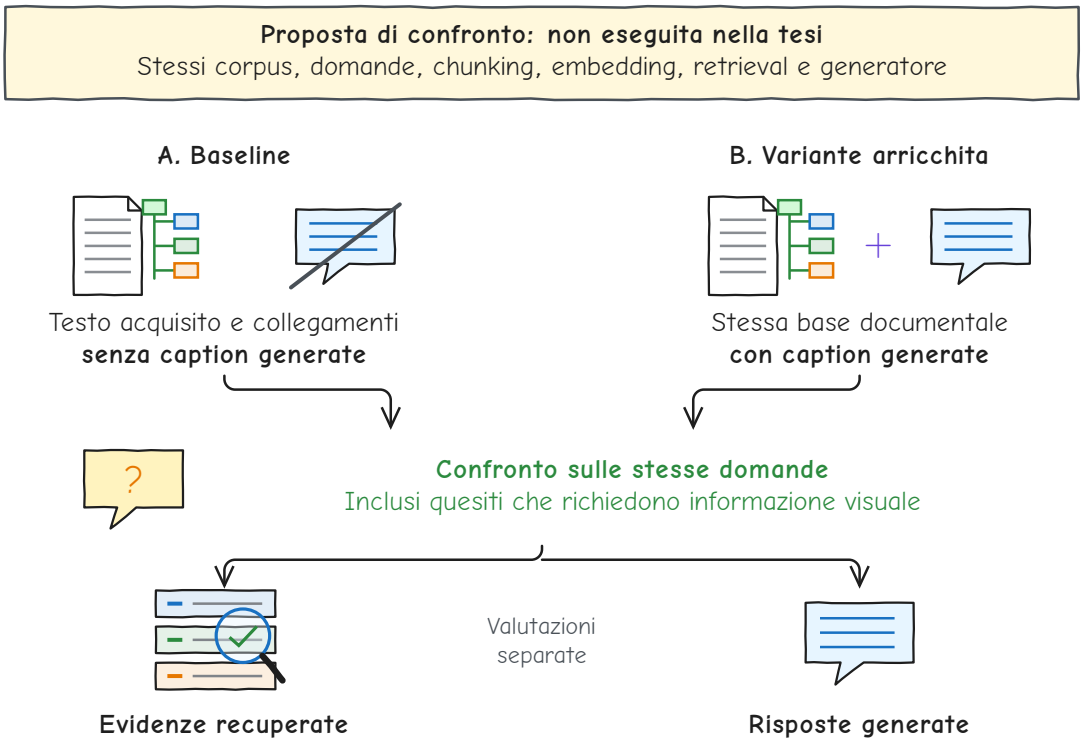


Figura 4.1: Disegno futuro per isolare il contributo delle caption al sistema RAG. Le due pagine rappresentano la stessa base documentale; il fumetto barrato indica l'assenza di caption generate nella baseline, non l'assenza di risposte del sistema. Le condizioni differiscono soltanto per l'arricchimento documentale e richiedono valutazioni separate di evidenze e risposte. Non sono mostrati risultati né effetti sulle prestazioni degli operatori.

Nota sull'utilizzo dei modelli di linguaggio

Nel corso del lavoro ho utilizzato OpenAI Codex, tramite l'estensione per Visual Studio Code, e Claude Code di Anthropic come assistenti nelle attività di sviluppo software. Li ho impiegati soprattutto per mettere in discussione le mie scelte progettuali, confrontare soluzioni alternative, individuare criticità, orientare la ricerca di documentazione tecnica da approfondire, analizzare errori e supportare la preparazione dei test. Li ho usati inoltre per controllare la coerenza tra codice, test, documentazione e artefatti sperimentali. Il loro contributo è stato particolarmente utile nei refactoring più estesi, resi necessari dalla progressiva comprensione del problema e dalla conseguente revisione dell'architettura della pipeline.

Nella scrittura della tesi ho utilizzato gli stessi strumenti come revisori critici, chiedendo loro di individuare passaggi poco chiari, incoerenze tra metodo, risultati e conclusioni, affermazioni non sufficientemente sostenute e possibili punti deboli dell'argomentazione. Li ho inoltre usati come supporto per l'organizzazione del documento, la revisione linguistica e la gestione dei sorgenti LaTeX. I suggerimenti ricevuti sono stati valutati e rielaborati sulla base del codice, degli artefatti sperimentali e delle fonti consultate.

Considerata l'ampiezza delle attività affrontate tra maggio e settembre 2026, ritengo che questi strumenti abbiano avuto un ruolo importante nel rendere sostenibile il lavoro entro il tempo disponibile. Senza il loro contributo, sarebbe stato difficilmente realistico raggiungere nello stesso periodo un livello analogo di sviluppo, sperimentazione, revisione e documentazione. La scelta di intraprendere questa tesi è nata anche dal desiderio di mettermi alla prova nell'uso consapevole di tecnologie emergenti e di sperimentarne direttamente potenzialità e limiti.

L'impiego del modello multimodale locale `qwen3.5:4b` tramite Ollama per la generazione delle descrizioni delle immagini costituisce invece parte del metodo sperimentale della tesi. Configurazione, attività svolta, controlli ed esiti sono descritti nel Capitolo 2 e discussi nel Capitolo 3.

Codice sorgente

Il codice sviluppato nell'ambito della tesi è disponibile pubblicamente su GitHub:

github.com/CometaSensitiva/industrial-manual-ingestion

Bibliografia

Amaram, Divija, Lu Gao, Gowtham Reddy Gudla e Tejaswini Sanjay Katale (2026). «Developing an AI Assistant for Knowledge Management and Workforce Training in State DOTs». In: *Electronics* 15.1, 217. DOI: 10.3390/electronics15010217.

Amugongo, Lameck Mbangula, Pietro Mascheroni, Steven Brooks, Stefan Doering e Jan Seidel (2025). «Retrieval augmented generation for large language models in healthcare: A systematic review». In: *PLOS Digital Health* 4.6, e0000877. DOI: 10.1371/journal.pdig.0000877.

Auer, Christoph, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Nikolaos Livathinos, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, Fabian Lindlbauer, Kasper Dinkla, Valery Weber, Lucas Morin, Ingmar Meijer, Viktor Kuropiatnyk e Peter W. J. Staar (19 ago. 2024). *Docling Technical Report*. arXiv: 2408.09869v1 [cs.CL].

Charan, Kamal V., Maneesh Singh, Surender Redhu e Gunjan Soni (2026). «A framework for automating failure modes and effects analysis (FMEA) using large language models (LLMs) and retrieval-augmented generation (RAG)». In: *International Journal of System Assurance Engineering and Management*. DOI: 10.1007/s13198-026-03171-6.

Cho, Seongwoo, Jongsu Park e Jumyung Um (2026). «On-Premise Multimodal AI Assistance for Operator-in-the-Loop Diagnosis in Machine Tool Mechatronic Systems». In: *Applied Sciences* 16.7, 3166. DOI: 10.3390/app16073166.

Colabianchi, Silvia, Francesco Costantino e Nicolò Sabetta (2024). «Assessment of a large language model based digital intelligent assistant in assembly manufacturing». In: *Computers in Industry* 162, 104129. DOI: 10.1016/j.compind.2024.104129.

Knollmeyer, Simon, Oğuz Caymazer e Daniel Grossmann (2025). «Document GraphRAG: Knowledge Graph Enhanced Retrieval Augmented Generation for Document Question Answering Within the Manufacturing Domain». In: *Electronics* 14.11, 2102. DOI: 10.3390/electronics14112102.

Krainovskikh, Veronika e Timur Samigulin (2026). «Domain-Adapted Retrieval-Augmented Generation for Technical Documentation: Enhancing Reliability and Faithfulness in Technical QA». In: *IEEE Access* 14, pp. 36016–36024. DOI: 10.1109/ACCESS.2026.3668910.

Lee, Kiho, Sumi Yang, Jaeyeong Jeong, Yongjoon Lee e Dongkyoo Shin (2025). «Enhancing Security and Applicability of Local LLM-Based Document Retrieval Systems in Smart Grid Isolated Environments». In: *Electronics* 14.17, 3407. DOI: 10.3390/electronics14173407.

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel e Douwe Kiela (2020). «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks». In: *Advances in Neural Information Processing Systems*. Vol. 33, pp. 9459–9474.

Ludwig, Heiner, Thorsten Schmidt e Mathias Kühn (2025). «An ontology-based retrieval augmented generation procedure for a voice-controlled maintenance assistant». In: *Computers in Industry* 169, 104289. DOI: 10.1016/j.compind.2025.104289.

Nam, Yerin, Hansun Choi, Jonggeun Choi e Hyukjin Kwon (2025). «LoRA-Tuned Multimodal RAG System for Technical Manual QA: A Case Study on Hyundai Staria». In: *Applied Sciences* 15.15, 8387. DOI: 10.3390/app15158387.

Qwen Team (29 apr. 2025). *Qwen3: Think Deeper, Act Faster*. URL: https://qwenlm.github.io/blog/qwen3/ (visitato il giorno 15/09/2026).

Riedler, Monica e Stefan Langer (29 ott. 2024). *Beyond Text: Optimizing RAG with Multimodal Inputs for Industrial Applications*. arXiv: 2410.21943v1 [cs.CL].

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser e Illia Polosukhin (2017). «Attention Is All You Need». In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 5998–6008.